ACTIVIDAD LABORATORIO NO.2
TRABAJO. EJERCICIOS DE MÁQUINAS DE TURING

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING ROGERIO ORLANDO BELTRAN CASTRO

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
01 DE MARZO
2026

TABLA DE CONTENIDO

TABLA DE CONTENIDO 2
INTRODUCCIÓN 3
DESARROLLO ACTIVIDAD 3
1. Ejercicio 1: Máquina de Turing para Número Par de Unos 4
1.1 Descripción del Problema 4
1.2 Estrategia de la Máquina 4
1.3 Definición Formal 5
1.4 Tabla de Transiciones 5
1.5 Diagrama de Estados 6
1.6 Implementación en JFLAP 7
2. Ejercicio 2: Máquina de Turing para Cadenas 1^+0^* 15
2.1 Descripción del Problema 15
2.2 Estrategia de la Máquina 16
2.3 Definición Formal y Tabla de Transiciones 16
2.4 Tabla de Transiciones 17
2.5 Implementación en JFLAP 19
CONCLUSIONES DE LA ACTIVIDAD 28
BIBLIOGRAFÍA 29
AGRADECIMIENTO 29

INTRODUCCIÓN

En el contexto de la teoría de la computación y los fundamentos de la informática, las Máquinas de Turing representan uno de los modelos más importantes para comprender el concepto de computabilidad y los límites de los sistemas algorítmicos. Estas máquinas constituyen un modelo abstracto capaz de simular cualquier proceso computacional, convirtiéndose en una herramienta esencial para el estudio formal de lenguajes, algoritmos y problemas decidibles o recursivamente enumerables.

El presente trabajo desarrolla un laboratorio práctico utilizando la herramienta JFLAP, un software educativo ampliamente empleado en el ámbito académico para la construcción y simulación de autómatas, gramáticas formales y Máquinas de Turing. Mediante esta herramienta es posible visualizar paso a paso el comportamiento de las máquinas, verificar su funcionamiento y analizar el proceso de aceptación o rechazo de cadenas dentro de un lenguaje determinado.

En esta actividad se abordó el diseño e implementación de dos Máquinas de Turing de una sola cinta, con alfabeto de entrada $\Sigma = \{0,1\}$ y alfabeto de cinta $\Gamma = \{0,1,\square\}$. Cada máquina fue construida para reconocer un lenguaje específico mediante una estrategia claramente definida de estados y transiciones.

En el primer ejercicio se diseñó una Máquina de Turing que acepta únicamente aquellas cadenas cuyo número de símbolos “1” es par. La estrategia consiste en alternar entre estados que representan la paridad actual (par o impar) cada vez que se lee un símbolo “1”, mientras que los símbolos “0” no modifican dicha condición. Al finalizar la lectura de la cadena (al encontrar el símbolo blanco), la máquina acepta si se encuentra en el estado correspondiente a una cantidad par de unos, y rechaza en caso contrario.

En el segundo ejercicio se construyó una Máquina de Turing que acepta exclusivamente cadenas formadas por una secuencia de uno o más unos seguida de cero o más ceros, excluyendo la cadena vacía. Este lenguaje puede expresarse formalmente como $1^{+}0^{*}$. La estrategia implementada consiste en verificar que la cadena inicie obligatoriamente con al menos un “1”, permitir la lectura de más unos consecutivos, y posteriormente aceptar únicamente si los símbolos siguientes son ceros hasta llegar al blanco. Cualquier aparición de un “1” después de haber leído un “0”, o el inicio con “0” o cadena vacía, conduce al rechazo.

El objetivo principal de este laboratorio es comprender de manera práctica el funcionamiento interno de las Máquinas de Turing, desarrollando la capacidad de definir estados, transiciones y estrategias algorítmicas dentro de JFLAP. Para validar cada diseño, se realizaron simulaciones con cadenas aceptadas y rechazadas, comprobando el correcto comportamiento de las máquinas construidas.

Este estudio permite reforzar la comprensión de los lenguajes formales y de los modelos abstractos de computación, evidenciando cómo una Máquina de Turing puede reconocer propiedades específicas de cadenas mediante reglas bien definidas. Asimismo, demuestra la utilidad de herramientas de simulación como JFLAP para fortalecer el aprendizaje práctico de conceptos fundamentales relacionados con computabilidad y teoría de lenguajes.

DESARROLLO ACTIVIDAD

A continuación, se presenta el desarrollo paso a paso del laboratorio de Máquinas de Turing utilizando la herramienta JFLAP. El proceso fue abordado de manera estructurada, construyendo cada máquina de forma progresiva y verificando su funcionamiento mediante simulaciones con diferentes cadenas de prueba. En las siguientes secciones se documentan las decisiones de diseño adoptadas, la definición de estados y transiciones, así como los resultados obtenidos durante la ejecución y validación de cada ejercicio.

En primer lugar, se describe la construcción de una Máquina de Turing de una sola cinta capaz de aceptar únicamente aquellas cadenas cuyo número de símbolos “1” es par. Se detalla la estrategia utilizada para controlar la paridad mediante la alternancia de estados que representan si la cantidad de unos leídos hasta el momento es par o impar. Asimismo, se explica el criterio de aceptación al finalizar la lectura de la cadena, verificando la condición requerida al encontrar el símbolo blanco en la cinta.

Posteriormente, se desarrolla una segunda Máquina de Turing diseñada para aceptar exclusivamente cadenas que pertenezcan al lenguaje 1^+0^*, es decir, cadenas formadas por uno o más unos seguidos de cero o más ceros, excluyendo la cadena vacía. En este caso, se expone la estrategia empleada para garantizar que la cadena inicie obligatoriamente con al menos un "1", permitir la lectura de unos consecutivos y, una vez iniciada la secuencia de ceros, impedir la aparición posterior de nuevos unos.

Cada etapa del laboratorio incluye la verificación mediante ejemplos de cadenas aceptadas y rechazadas, lo que permite comprobar el correcto funcionamiento de las máquinas diseñadas y reforzar la comprensión práctica del modelo computacional propuesto por Alan Turing. De esta manera, el desarrollo no solo evidencia la correcta implementación técnica en JFLAP, sino también la comprensión conceptual de los criterios formales que definen cada lenguaje.

1. Ejercicio 1: Máquina de Turing para Número Par de Unos

1.1 Descripción del Problema

Se desea construir una Máquina de Turing (MT) determinista de una sola cinta cuyo alfabeto de entrada sea $\Sigma = \{0, 1\}$ y cuyo alfabeto de cinta sea $\Gamma = \{0, 1, \square\}$, donde el símbolo $\square$ representa el espacio en blanco que delimita el final de la cadena en la cinta. La máquina deberá entrar en estado de aceptación únicamente cuando el número total de símbolos "1" presentes en la cadena de entrada sea par, considerando como caso válido la cadena que contenga cero unos.

El problema consiste en diseñar un mecanismo formal que permita a la máquina recorrer la cadena de izquierda a derecha, analizar cada símbolo y determinar si la cantidad de unos leídos cumple la condición de paridad requerida. Es importante destacar que los símbolos "0" no influyen en la condición de aceptación, ya que únicamente se evalúa la cantidad de "1" en la cadena. Por tanto, la máquina debe ignorar los ceros en términos de cómputo de paridad, aunque necesariamente debe desplazarse sobre ellos durante el recorrido.

Desde el punto de vista formal, el lenguaje que se desea reconocer puede describirse como:

$$L = \{ w \in \{0,1\}^* \mid \text{el número de símbolos 1 en w es par} \}$$

Este lenguaje pertenece a la clase de los lenguajes regulares; sin embargo, el objetivo del ejercicio es implementarlo específicamente mediante una Máquina de Turing, reforzando así la comprensión del modelo computacional y su funcionamiento interno.

La máquina debe procesar la cadena completa hasta encontrar el símbolo blanco ($\square$), el cual indica el final de la entrada. Solo en ese momento podrá decidir si acepta o rechaza la cadena, dependiendo del estado en el que se encuentre.

1.2 Estrategia de la Máquina

La estrategia fundamental para resolver el problema consiste en controlar la paridad del número de símbolos "1" leídos durante el recorrido de la cinta. En lugar de realizar un conteo absoluto de los unos, lo cual implicaría un almacenamiento numérico explícito, se utiliza una técnica basada en el cambio alternado de estados internos. De esta manera, la máquina solo necesita recordar si la cantidad de unos leídos hasta el momento es par o impar.

El procedimiento consiste en recorrer la cinta de izquierda a derecha desde la primera posición de la cadena hasta encontrar el símbolo blanco ($\square$), que marca el final de la entrada. Durante este recorrido:

Cuando se lee un símbolo "0", la máquina permanece en el mismo estado, ya que los ceros no afectan la paridad.

Cuando se lee un símbolo "1", la máquina alterna entre los estados q_0 y q_1.

La máquina está compuesta por cuatro estados:

- q_0: representa que la cantidad de unos leídos hasta el momento es par.
- q_1: representa que la cantidad de unos leídos es impar.
- q_accept: estado final de aceptación.
- q_reject: estado final de rechazo.

El estado inicial es q_0, ya que antes de leer cualquier símbolo la cantidad de unos es cero, y cero es un número par.

Cuando la máquina encuentra el símbolo blanco (□):

- Si se encuentra en q_0, transita a q_accept.
- Si se encuentra en q_1, transita a q_reject.

Desde el punto de vista operativo, el movimiento de la cabeza lectora es siempre hacia la derecha (R), excepto en el momento de leer el blanco, donde se utiliza el movimiento S (Stay) para detener la máquina en el estado correspondiente.

Formalmente, el comportamiento puede resumirse como:

- $\delta(q_0, 0) = (q_0, 0, R)$
- $\delta(q_0, 1) = (q_1, 1, R)$
- $\delta(q_1, 0) = (q_1, 0, R)$
- $\delta(q_1, 1) = (q_0, 1, R)$
- $\delta(q_0, \square) = (q_accept, \square, S)$
- $\delta(q_1, \square) = (q_reject, \square, S)$

Esta estrategia demuestra cómo los estados de una Máquina de Turing pueden funcionar como un mecanismo de memoria finita, permitiendo reconocer propiedades específicas de una cadena —en este caso, la paridad— sin necesidad de modificar la cinta ni utilizar símbolos auxiliares adicionales.

1.3 Definición Formal

La Máquina de Turing diseñada para reconocer cadenas con número par de unos se define formalmente como la séptupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$$

donde:

- $Q = \{ q_0, q_1, q_accept, q_reject \}$
- $\Sigma = \{0,1\}$
- $\Gamma = \{0,1,\square\}$
- q_0 es el estado inicial
- □ es el símbolo blanco
- $F = \{ q_accept \}$
- δ es la función de transición

En esta máquina:

- q_0 representa que la cantidad de unos leídos hasta el momento es par.
- q_1 representa que la cantidad de unos leídos es impar.

La aceptación no ocurre directamente en q_0, sino en el estado q_accept, al cual se transita cuando se alcanza el blanco estando en condición par.

1.4 Tabla de Transiciones

Estado actual	Símbolo leído	Símbolo escrito	Movimiento	Estado siguiente
q_0	0	0	R	q_0
q_0	1	1	R	q_1
q_0	□	□	S	q_accept
q_1	0	0	R	q_1
q_1	1	1	R	q_0
q_1	□	□	S	q_reject

El comportamiento general puede describirse así:

- Al leer un 0, la máquina permanece en el mismo estado, ya que los ceros no afectan la paridad.
- Al leer un 1, la máquina alterna entre $q0$ y $q1$. Al encontrar el símbolo blanco (□), la decisión depende del estado actual:
  - Si está en $q0$, acepta.
  - Si está en $q1$, rechaza.

1.5 Diagrama de Estados

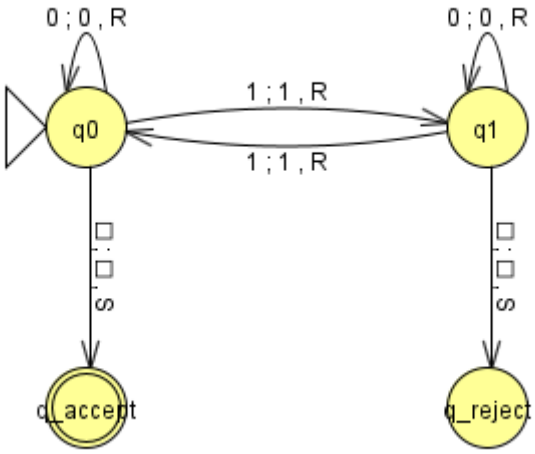


Figura 1. Diagrama de la Máquina de Turing que acepta cadenas con número par de unos. El estado q_accept (doble circunferencia) es el estado de aceptación, mientras que q_reject es el estado de rechazo.

El diagrama refleja un mecanismo de alternancia entre estados:

- Desde $q0$, al leer un “1”, se pasa a $q1$.
- Desde $q1$, al leer un “1”, se retorna a $q0$.
- Los símbolos “0” no provocan cambio de estado.

1.5.1 Ejemplos de trazas

Con el fin de verificar el comportamiento teórico de la Máquina de Turing diseñada, se presentan a continuación ejemplos de ejecución descritos formalmente mediante la secuencia de estados recorridos.

1.5.1.1 Ejemplo 1: Cadena "110" (Aceptada)

La cadena contiene dos símbolos “1”, por lo tanto, la cantidad es par y debe ser aceptada.

Secuencia de estados:

- $q_0 \rightarrow(1)\rightarrow q_1$
- $q_1 \rightarrow(1)\rightarrow q_0$
- $q_0 \rightarrow(0)\rightarrow q_0$
- $q_0 \rightarrow(\square)\rightarrow$ q_accept

Explicación:

- La máquina inicia en q_0 (estado par).
- Lee el primer “1” y cambia a q_1 (impar).
- Lee el segundo “1” y regresa a q_0 (par).
- Lee el “0” y permanece en q_0.
- Al encontrar el símbolo blanco (□) estando en q_0, transita a q_accept.

Por lo tanto, la cadena es aceptada.

1.5.1.2 Ejemplo 2: Cadena "1" (Rechazada)

La cadena contiene un solo símbolo “1”, por lo tanto, la cantidad es impar y debe ser rechazada.

Secuencia de estados:

- $q_0 \rightarrow(1)\rightarrow q_1$
- $q_1 \rightarrow(\square)\rightarrow$ q_reject

Explicación:

- La máquina inicia en q_0.
- Lee el símbolo “1” y cambia a q_1 (impar).
- Al encontrar el símbolo blanco (□) estando en q_1, transita a q_reject.

Por lo tanto, la cadena es rechazada.

1.6 Implementación en JFLAP

Para validar el diseño teórico de la Máquina de Turing que reconoce cadenas con número par de unos, se realizó su implementación en el software JFLAP, utilizando una Máquina de Turing de una sola cinta.

1.6.1 Creación de la máquina

En JFLAP se seleccionó la opción:

File → New → Turing Machine

Posteriormente, se construyeron los estados definidos en la especificación formal:

- q_0: estado inicial.
- q_1: estado que representa una cantidad impar de unos.
- q_accept: estado final de aceptación (doble circunferencia).
- q_reject: estado de rechazo.

El estado q_0 fue marcado únicamente como estado inicial, mientras que q_accept fue marcado como estado final de aceptación, de acuerdo con la definición formal:

F = {q_accept}

1.6.2 Definición de transiciones en JFLAP

Las transiciones fueron agregadas utilizando la función “Add Transition”, ingresando los valores en el formato:

símbolo leído ; símbolo escrito ; movimiento

Las transiciones implementadas fueron:

- $\delta(q_0, 0) = (q_0, 0, R)$
- $\delta(q_0, 1) = (q_1, 1, R)$
- $\delta(q_1, 0) = (q_1, 0, R)$
- $\delta(q_1, 1) = (q_0, 1, R)$
- $\delta(q_0, \square) = (q_accept, \square, S)$
- $\delta(q_1, \square) = (q_reject, \square, S)$

Para el caso del símbolo blanco (□):

- Desde q_0, al leer □, la máquina se detiene en estado de aceptación.
- Desde q_1, al leer □, la máquina se detiene sin aceptar (rechazo).

En JFLAP esto se implementa utilizando el movimiento S (Stay) o simplemente dejando que no existan transiciones adicionales desde el estado final.

1.6.3 Funcionamiento durante la simulación

La simulación se realizó mediante la opción:

Input → Step o Input → Fast Run

Durante la ejecución, se observa que:

- La cabeza lectora se desplaza siempre hacia la derecha.
- Cada vez que se lee un “1”, el estado alterna entre q_0 y q_1.
- Los símbolos “0” no generan cambio de estado.
- Al llegar al símbolo blanco (□), la máquina toma la decisión final dependiendo del estado actual.

1.6.4 Ejemplos de validación en JFLAP

A continuación, dos ejemplos que demuestran la correcta ejecución del código, uno aprobando y el otro rechazando:

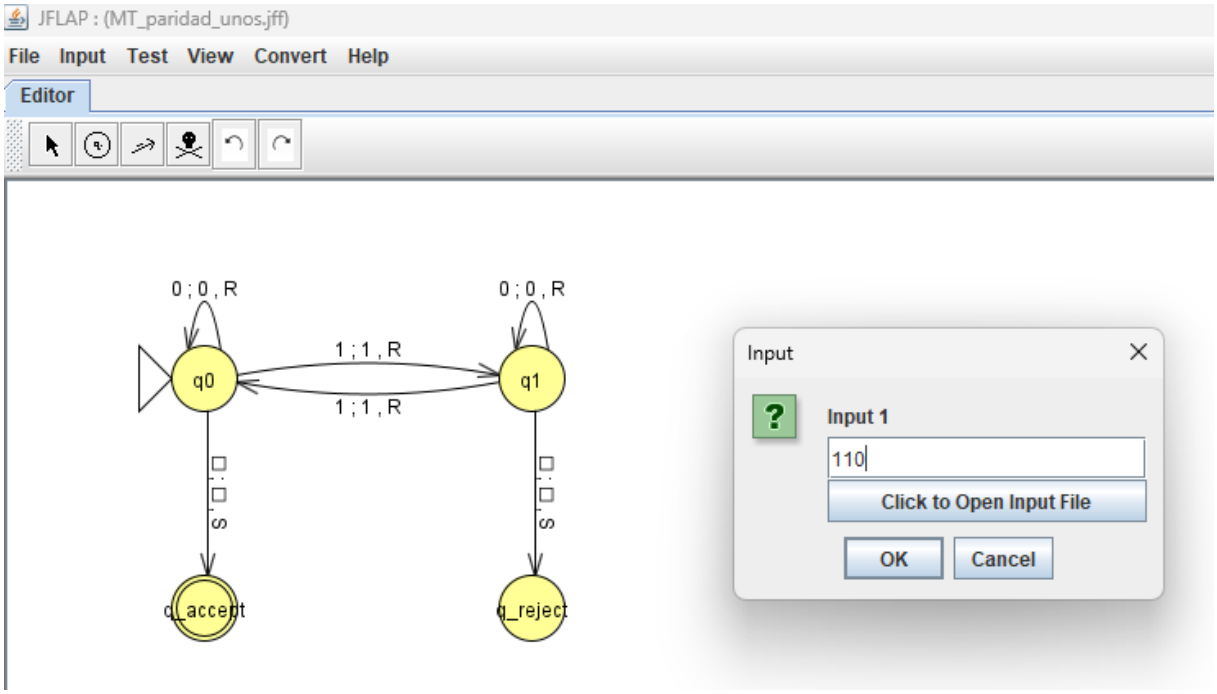
1.6.4.1 Ejemplo No. 1:

Para la cadena 110:

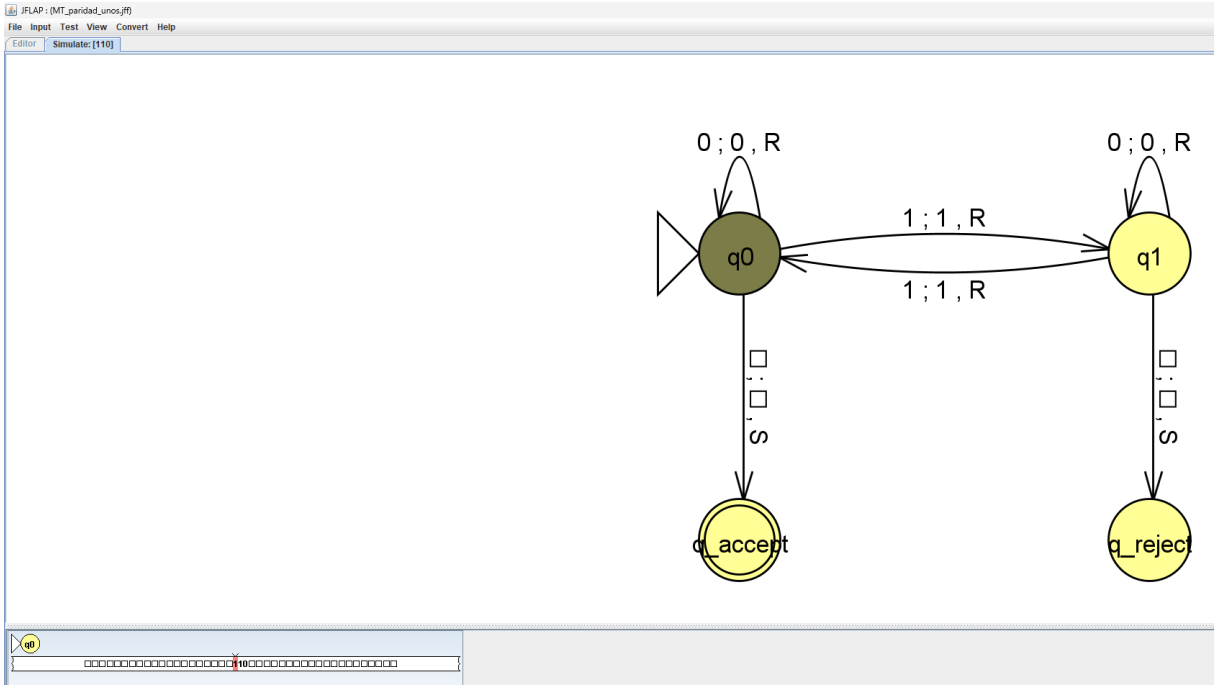
- q_0 lee 1 → pasa a q_1
- q_1 lee 1 → pasa a q_0
- q_0 lee 0 → permanece en q_0
- q_0 lee □ → ACEPTA

A continuación, las imágenes respectivas de proceso:

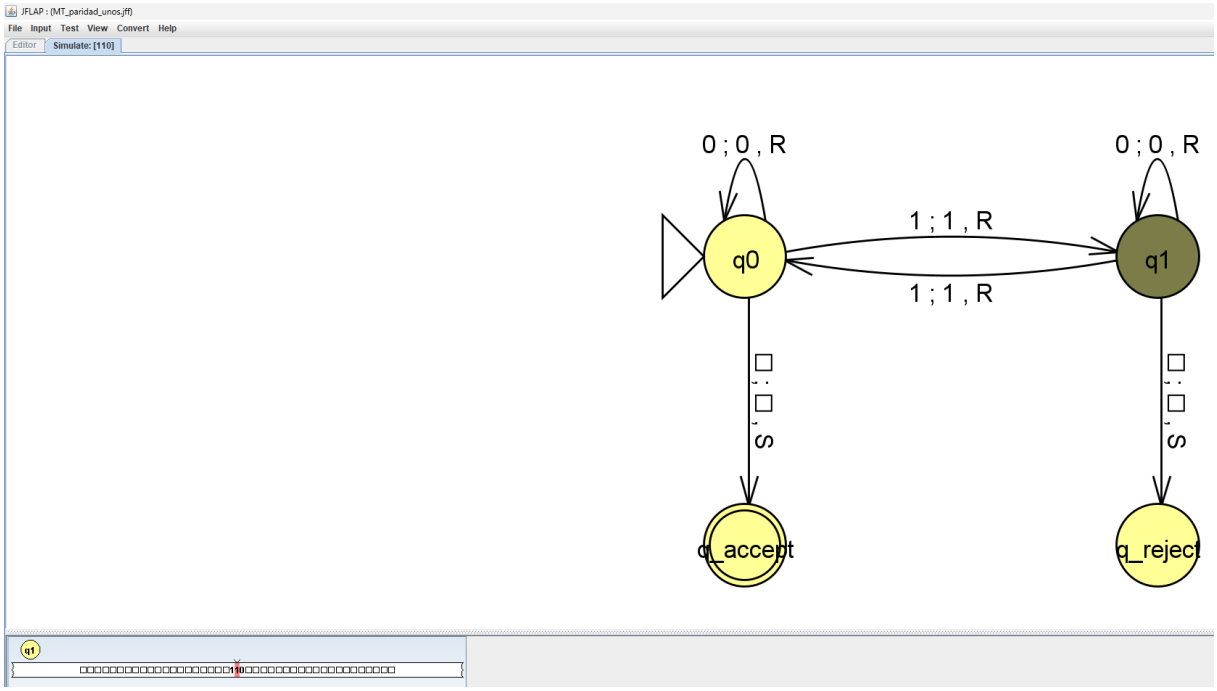
1.6.4.1.1 Imagen 1:



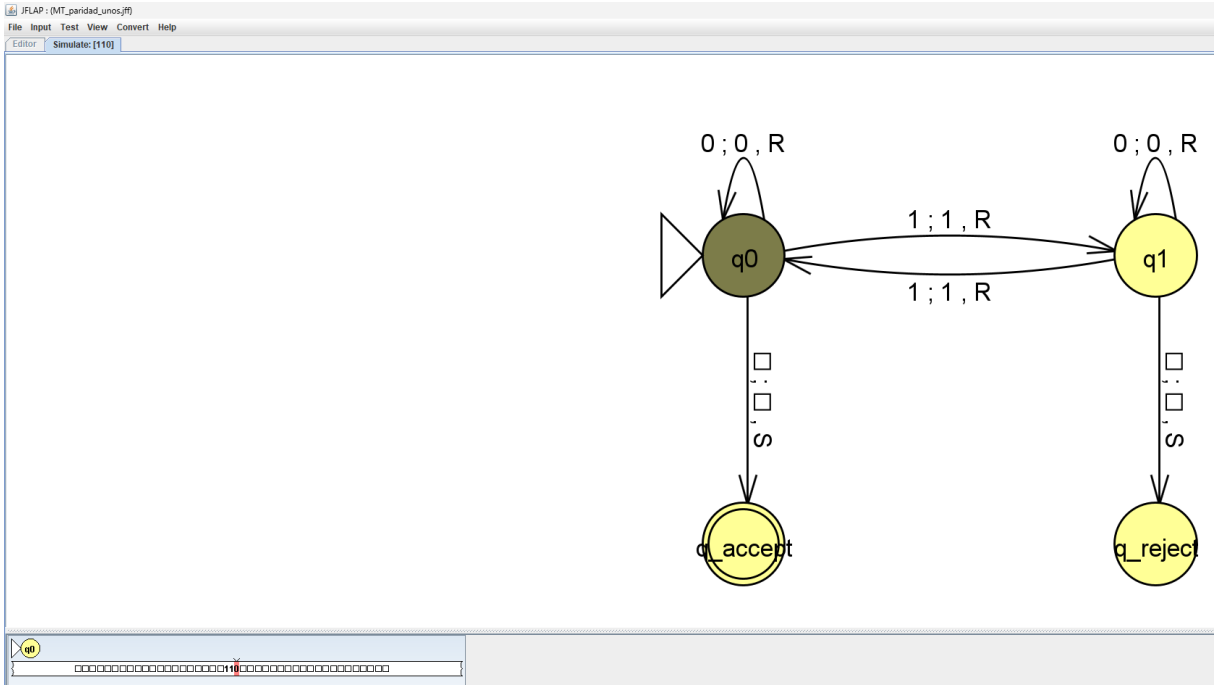
1.6.4.1.2 Imagen 2:



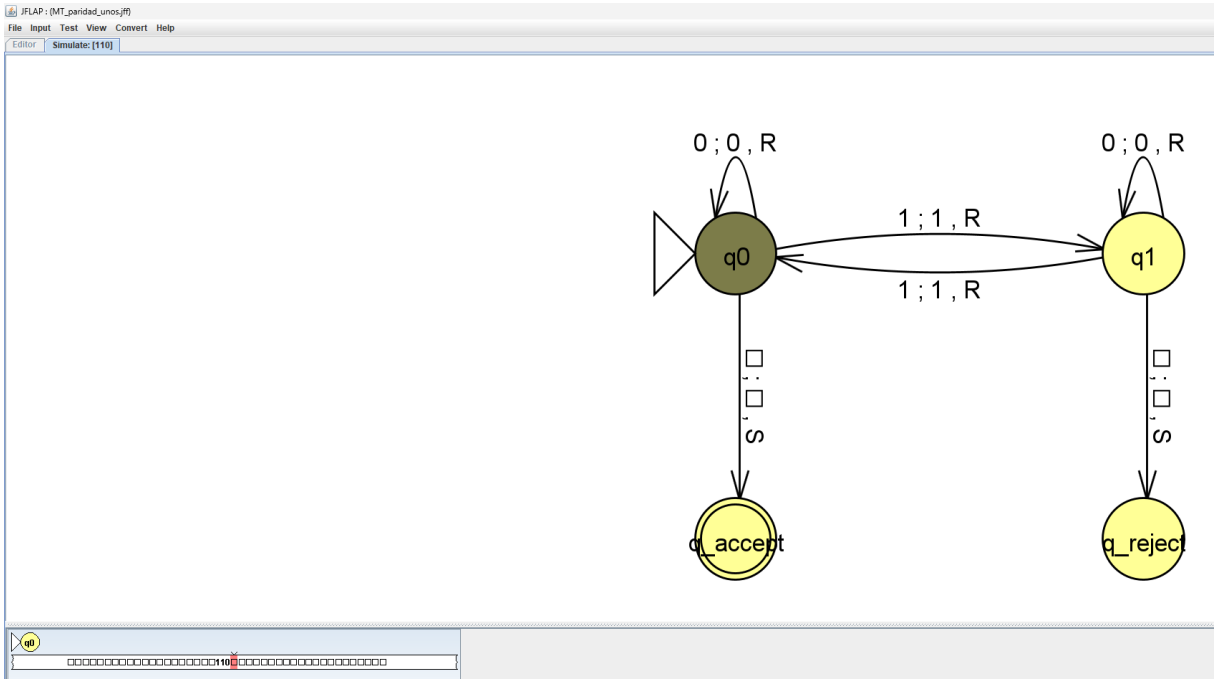
1.6.4.1.3 Imagen 3:



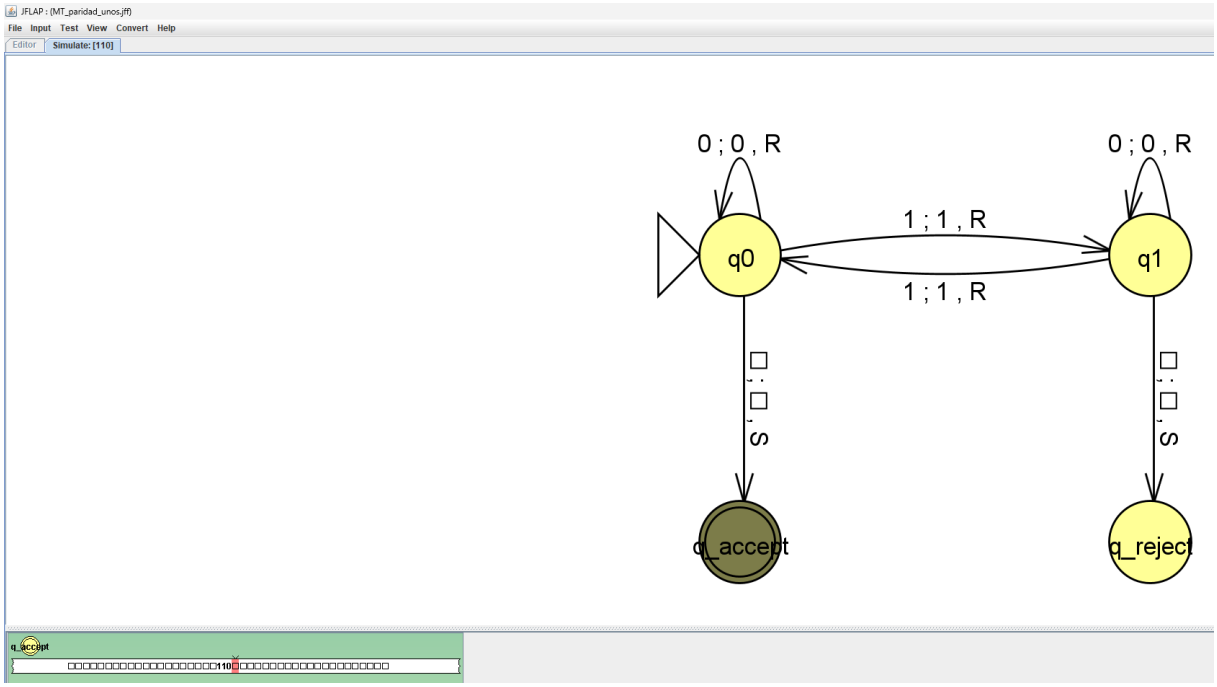
1.6.4.1.4 Imagen 4:



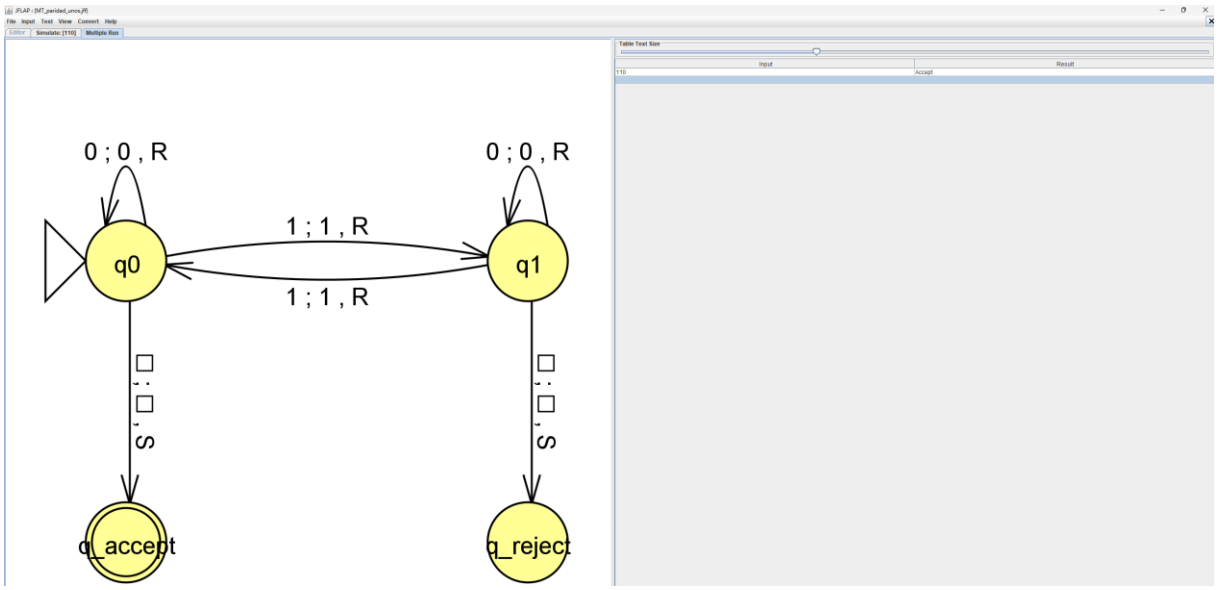
1.6.4.1.5 Imagen 5:



1.6.4.1.6 Imagen 6:



1.6.4.1.7 Imagen 7:



1.6.4.2 Ejemplo No. 2:

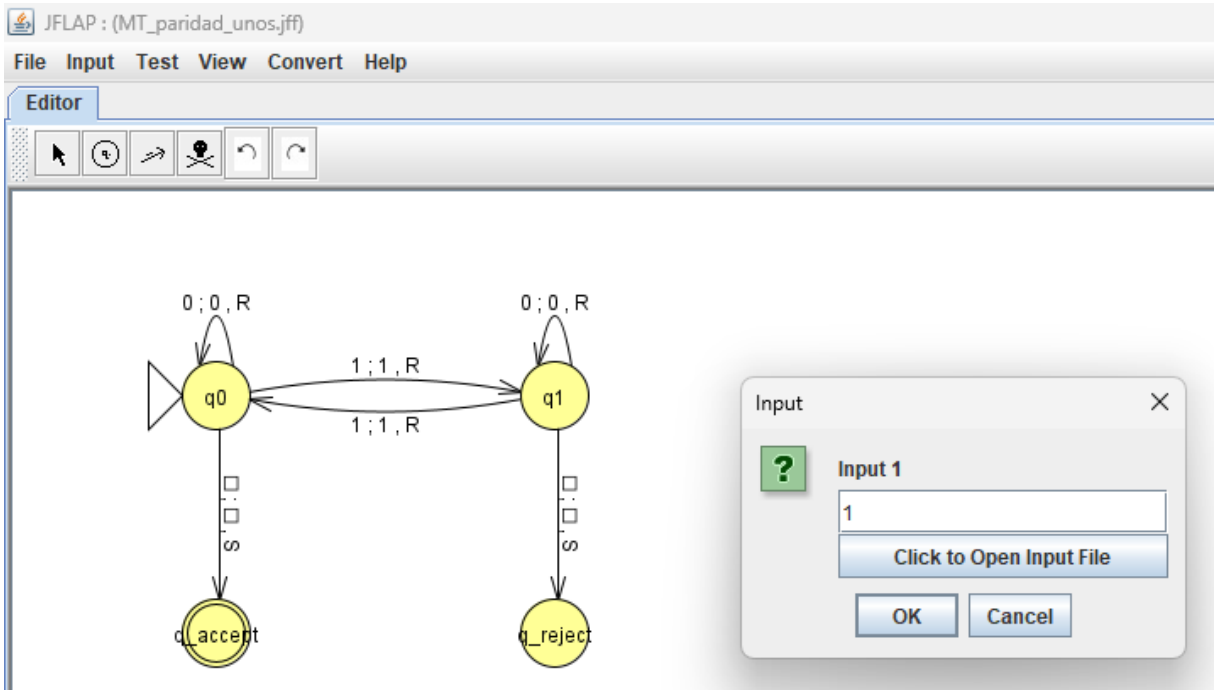
Para la cadena 1:

- q_0 lee 1 → pasa a q_1
- q_1 lee □ → RECHAZA

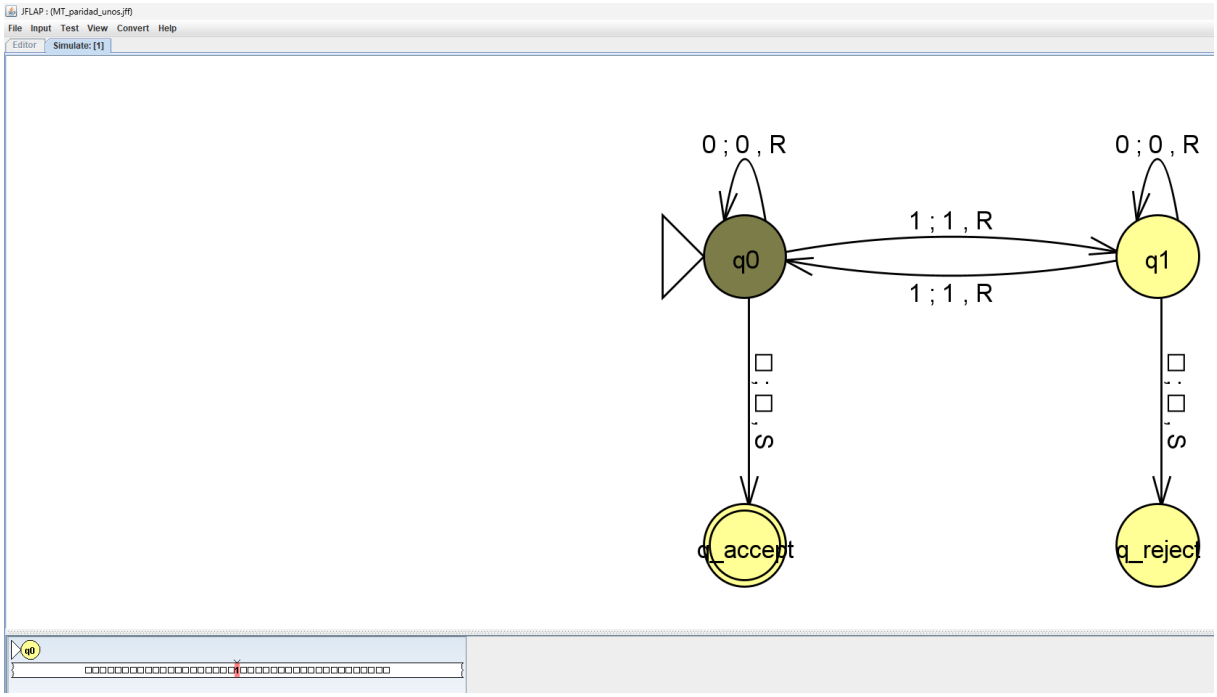
Las simulaciones confirmaron que la implementación en JFLAP coincide exactamente con el comportamiento esperado según la definición formal y la tabla de transiciones.

A continuación, las imágenes respectivas de proceso:

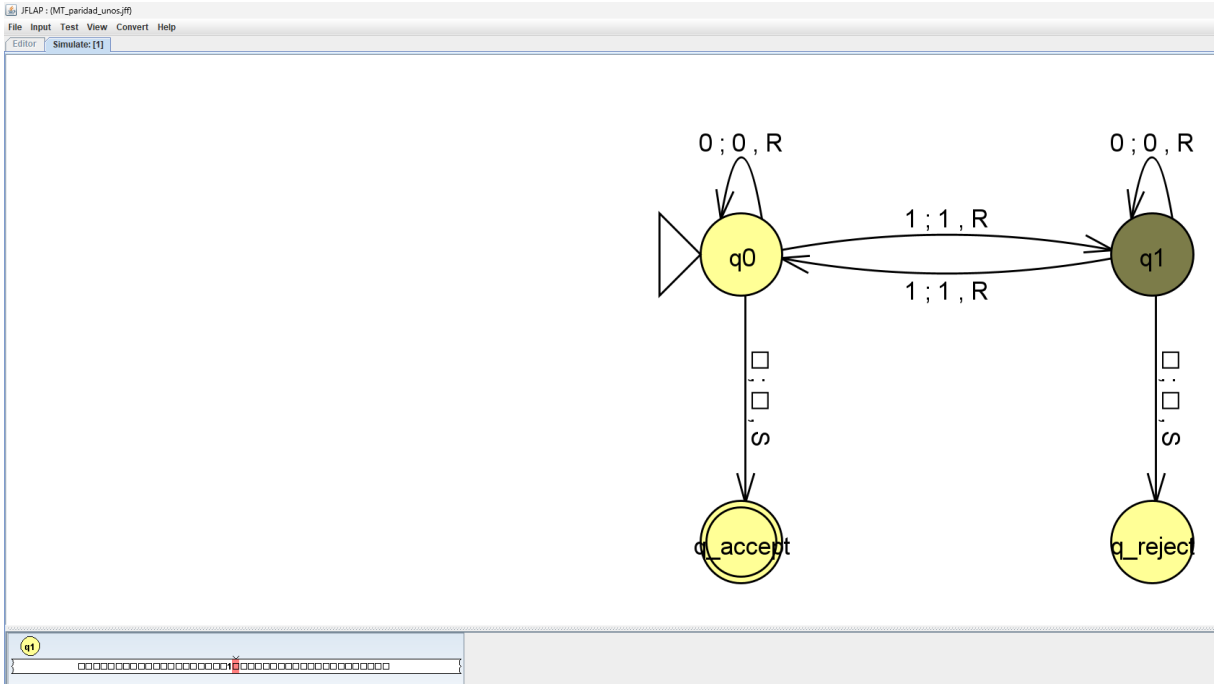
1.6.4.2.1 Imagen 1:



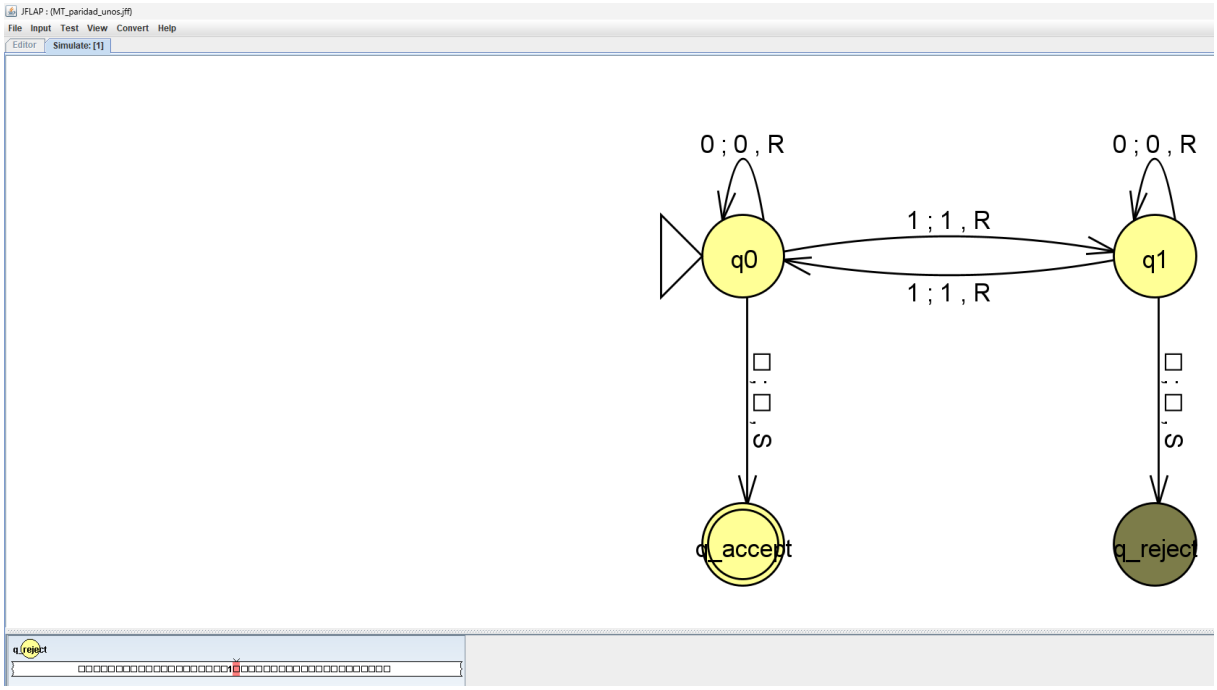
1.6.4.2.2 Imagen 2:



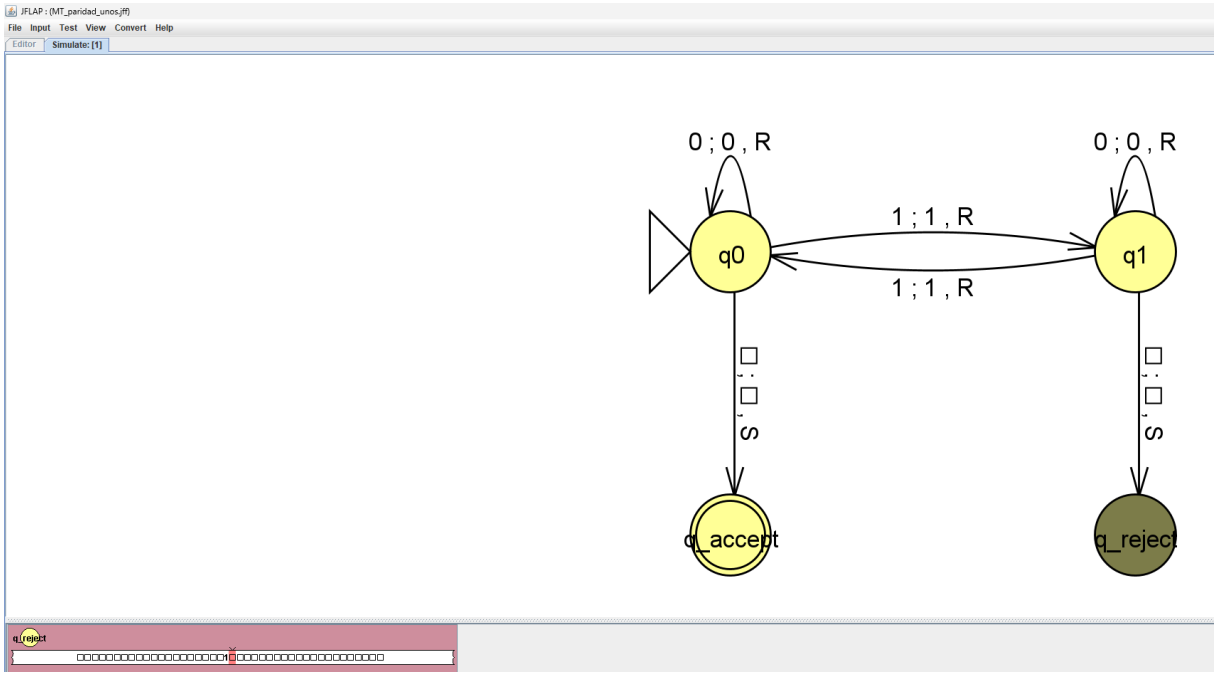
1.6.4.2.3 Imagen 3:



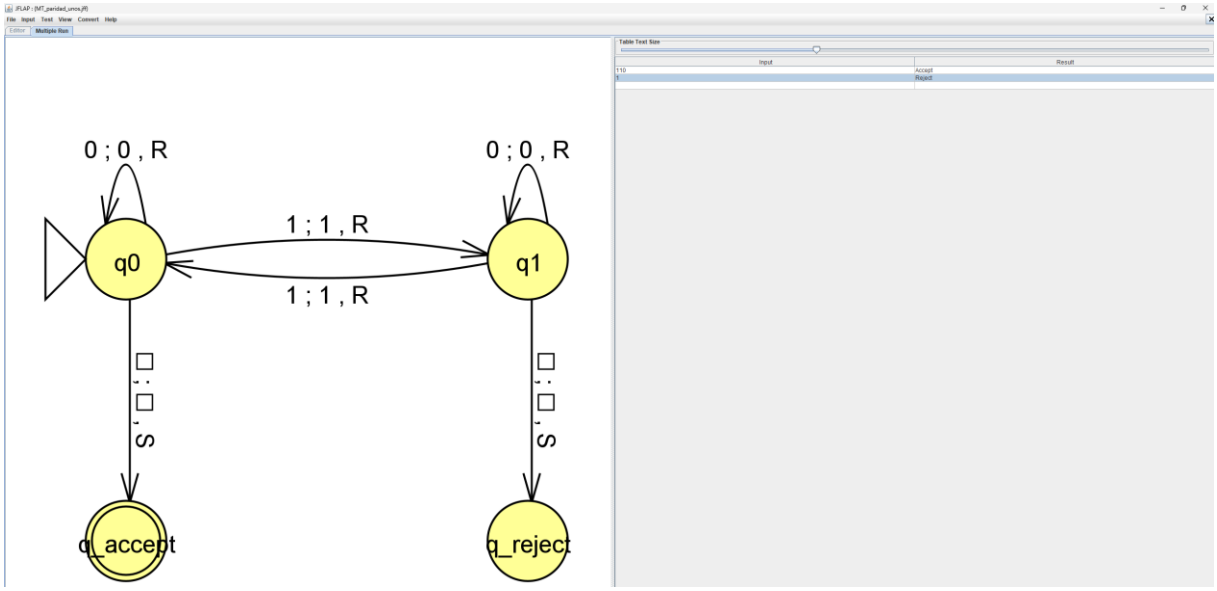
1.6.4.2.4 Imagen 4:



1.6.4.2.5 Imagen 5:



1.6.4.2.6 Imagen 6:



2. Ejercicio 2: Máquina de Turing para Cadenas $1^{+}0^{*}$

2.1 Descripción del Problema

Se desea construir una Máquina de Turing (MT) determinista de una sola cinta cuyo alfabeto de entrada sea Σ = {0, 1} y cuyo alfabeto de cinta sea Γ = {0, 1, □}, donde el símbolo □ representa el blanco que delimita el final de la cadena en la cinta.

La máquina deberá aceptar exclusivamente aquellas cadenas que estén formadas por uno o más símbolos “1” seguidos de cero o más símbolos “0”, es decir, cadenas que pertenezcan al lenguaje:

$$\mathbf{L} = \{\mathbf{1^n 0^m} \mid \mathbf{n \geq 1, m \geq 0}\}$$

Esto significa que:

- La cadena debe comenzar obligatoriamente con al menos un “1”.
- Puede contener varios “1” consecutivos.
- Después del último “1”, pueden aparecer cero o más “0”.
- Una vez que aparece un “0”, no puede volver a aparecer un “1”.
- La cadena vacía (n = 0) queda excluida explícitamente.

En términos estructurales, la cadena debe respetar estrictamente el patrón:

2.1.1 bloque de unos → bloque de ceros

Ejemplos de cadenas aceptadas:

- "1"
- "10"
- "100"
- "110"
- "1100"

Ejemplos de cadenas rechazadas:

- "" (cadena vacía)
- "0"
- "01"
- "101"

- "001"

El objetivo es diseñar una Máquina de Turing que verifique esta estructura mediante un recorrido secuencial de izquierda a derecha y que determine la aceptación únicamente si la cadena cumple exactamente con el patrón definido.

2.2 Estrategia de la Máquina

La estrategia consiste en realizar un único recorrido sobre la cinta, verificando que la cadena respete el orden estructural requerido. La máquina no necesita modificar los símbolos, sino únicamente analizarlos y cambiar de estado según el bloque que esté procesando.

La máquina se divide conceptualmente en tres fases:

- Verificación del inicio correcto (al menos un “1”).
- Procesamiento del bloque de unos.
- Procesamiento del bloque de ceros.

Los estados cumplen las siguientes funciones:

2.2.1 q_0 (estado inicial)

Es el estado donde comienza la máquina. En este estado se verifica que la cadena inicie con un “1”.

- Si se lee un “1”, la máquina transita a q_1.
- Si se lee un “0”, la cadena se rechaza inmediatamente.
- Si se lee el blanco (□), significa que la cadena está vacía y se rechaza.

Este estado garantiza que $n \geq 1$.

2.2.2 q_1 (fase de unos)

En este estado la máquina procesa el bloque inicial de unos.

- Si lee otro “1”, permanece en q_1.
- Si lee un “0”, significa que comienza el bloque de ceros y transita a q_2.
- Si lee el blanco directamente (cadena compuesta solo por unos), también puede aceptarse, ya que $m \geq 0$.

Este estado garantiza que todos los “1” estén agrupados al inicio.

2.2.3 q_2 (fase de ceros)

En este estado la máquina procesa el bloque final de ceros.

- Si lee un “0”, permanece en q_2.
- Si lee el blanco (□), la cadena es aceptada.
- Si lee un “1”, significa que la estructura se rompe (por ejemplo, “101”) y se transita al estado de rechazo.

Este estado asegura que después de comenzar el bloque de ceros no aparezcan más unos.

2.2.4 q_reject (estado de rechazo)

Es el estado sumidero. La máquina llega aquí cuando detecta que la cadena no respeta el patrón requerido. Una vez en este estado, la máquina se detiene sin aceptar.

2.3 Definición Formal y Tabla de Transiciones

La Máquina de Turing se define formalmente como:

$$\mathbf{M} = (\mathbf{Q}, \mathbf{\Sigma}, \mathbf{\Gamma}, \boldsymbol{\delta}, \mathbf{q0}, \square, \mathbf{F})$$

donde:

- Q = {q_0, q_1, q_2, q_accept, q_reject}
- Σ = {0,1}
- Γ = {0,1,□}
- q_0 es el estado inicial
- □ es el símbolo blanco
- F = {q_accept}

2.4 Tabla de Transiciones

Estado actual	Símbolo leído	Símbolo escrito	Movimiento	Estado siguiente
q_0	1	1	R	q_1
q_0	0	0	S	q_reject
q_0	□	□	S	q_reject
q_1	1	1	R	q_1
q_1	0	0	R	q_2
q_1	□	□	S	q_accept
q_2	0	0	R	q_2
q_2	□	□	S	q_accept
q_2	1	1	S	q_reject

El movimiento S (Stay) indica que la cabeza no se desplaza y la máquina se detiene en el estado correspondiente.
Este diseño garantiza que:

- La cadena no pueda comenzar con 0.
- No pueda ser vacía.
- No pueda aparecer un 1 después de un 0.
- Se acepte tanto 1^+ como 1^+0^*.

2.4.1 Diagrama de Estados

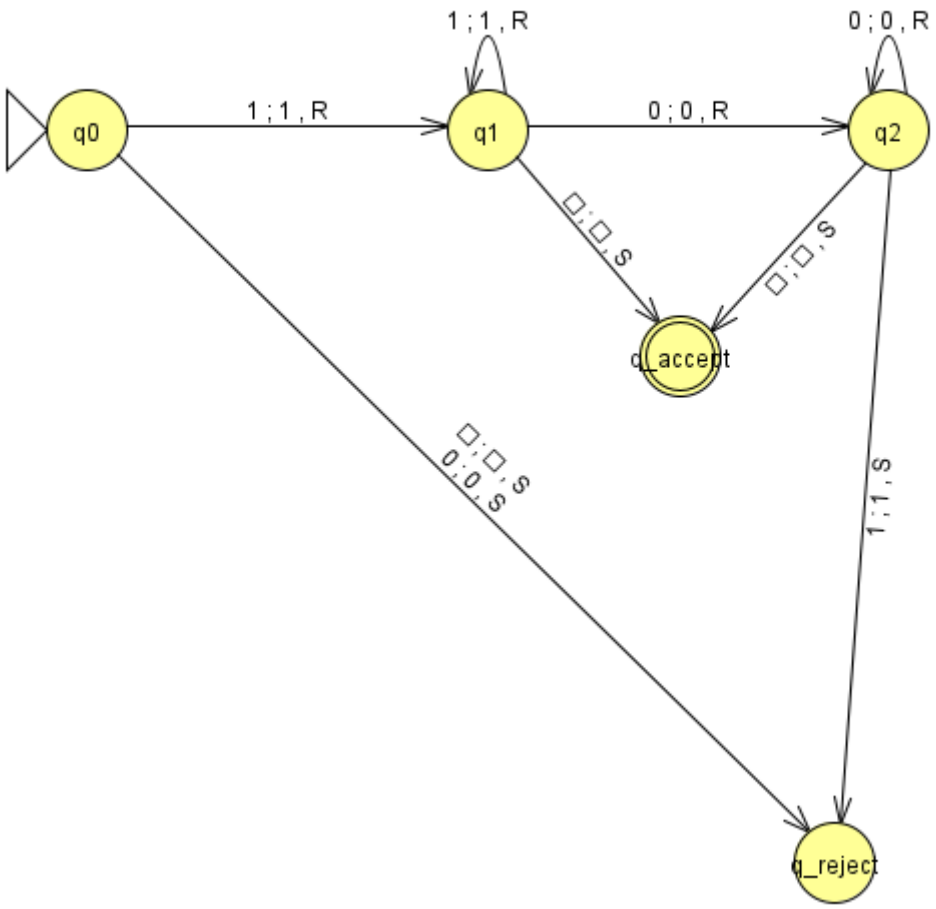


Figura 2. Diagrama de la Máquina de Turing que acepta cadenas de la forma $1 + 0^*$. El estado q_accept (doble circunferencia) es el estado de aceptación, mientras que q_reject es el estado de rechazo.

El diagrama representa claramente las tres fases del procesamiento de la cadena:

- q_0: estado inicial, donde se verifica que la cadena comience con al menos un símbolo “1”.
- q_1: estado encargado de procesar el bloque de unos.
- q_2: estado encargado de procesar el bloque de ceros.
- q_accept: estado final de aceptación.
- q_reject: estado final de rechazo.

Desde q_0, la máquina solo puede continuar si lee un “1”. Si la primera celda contiene un “0” o el símbolo blanco (□), la máquina transita inmediatamente a q_reject, ya que la cadena no cumple la condición n≥1.

En q_1, la máquina continúa leyendo unos. Si aparece un “0”, significa que comienza el bloque de ceros y la máquina pasa a q_2. Si aparece directamente el blanco (□), la cadena contiene únicamente unos y se acepta transitando a q_accept.

En q_2, la máquina procesa el bloque de ceros. Si continúa leyendo “0”, permanece en q_2. Si aparece el símbolo blanco, la cadena es válida y la máquina pasa a q_accept. Sin embargo, si en q_2 se lee un “1”, significa que el patrón fue violado (apareció un 1 después de un 0), por lo que la máquina transita a q_reject.

2.4.2 Ejemplos de Trazas

2.4.2.1 Ejemplo 1: Cadena "110" (Aceptada)

Secuencia de estados:

$$\mathbf{q^0} \rightarrow (\mathbf{1}) \rightarrow \mathbf{q^1} \rightarrow (\mathbf{1}) \rightarrow \mathbf{q^1} \rightarrow (\mathbf{0}) \rightarrow \mathbf{q^2} \rightarrow (\square) \rightarrow \mathbf{q_{accept}}$$

La cadena inicia con uno o más unos y posteriormente contiene ceros sin volver a aparecer un uno. Por lo tanto, pertenece al lenguaje y es aceptada.

2.4.2.2 Ejemplo 2: Cadena "101" (Rechazada)

Secuencia de estados:

$$\mathbf{q^0} \rightarrow (\mathbf{1}) \rightarrow \mathbf{q^1} \rightarrow (\mathbf{0}) \rightarrow \mathbf{q^2} \rightarrow (\mathbf{1}) \rightarrow \mathbf{q_{reject}}$$

El símbolo “1” aparece después de haber comenzado el bloque de ceros, lo cual viola la estructura $1 + 0^*$. La cadena es rechazada.

2.4.2.3 Ejemplo 3: Cadena "0" (Rechazada)

Secuencia de estados:

$$\mathbf{q^0} \rightarrow (\mathbf{0}) \rightarrow \mathbf{q_{reject}}$$

La cadena no comienza con al menos un “1”, por lo tanto, no cumple la condición n≥1 y es rechazada inmediatamente.

2.5 Implementación en JFLAP

Para validar el diseño teórico de la Máquina de Turing que reconoce cadenas de la forma $1 + 0^*$, se realizó su implementación en el software JFLAP, utilizando una Máquina de Turing determinista de una sola cinta.

2.5.1 Creación de la máquina

En JFLAP se seleccionó la opción:

File → New → Turing Machine

Posteriormente, se construyeron los estados definidos en la especificación formal:

- q_0: estado inicial.
- q_1: estado correspondiente a la fase de lectura de unos.
- q_2: estado correspondiente a la fase de lectura de ceros.
- q_accept: estado final de aceptación (doble circunferencia).
- q_reject: estado de rechazo.

El estado q_0 fue marcado únicamente como estado inicial, mientras que q_accept fue marcado como estado final de aceptación, de acuerdo con la definición formal:

F={q_accept}

2.5.2 Definición de transiciones en JFLAP

Las transiciones fueron agregadas utilizando la herramienta “Add Transition”, ingresando los valores en el formato:

símbolo leído ; símbolo escrito ; movimiento

Las transiciones implementadas fueron:

- $\delta(q_0, 1) = (q_1, 1, R)$
- $\delta(q_0, 0) = (q_reject, 0, S)$
- $\delta(q_0, \square) = (q_reject, \square, S)$

- $\delta(q_1, 1) = (q_1, 1, R)$
- $\delta(q_1, 0) = (q_2, 0, R)$
- $\delta(q_1, \square) = (q_accept, \square, S)$

- $\delta(q_2, 0) = (q_2, 0, R)$
- $\delta(q_2, 1) = (q_reject, 1, S)$
- $\delta(q_2, \square) = (q_accept, \square, S)$

En el estado q_reject no se definieron transiciones adicionales, ya que es un estado sumidero de rechazo.

2.5.3 Interpretación de las transiciones

- Desde q_0 solo es válido leer un "1".
- Si la cadena inicia con "0" o está vacía (□), se rechaza inmediatamente.
- En q_1 se procesan los unos iniciales.
- Si aparece un "0", se pasa a q_2 para comenzar la fase de ceros.
- En q_2 solo pueden leerse ceros hasta el final.
- Si aparece un "1" después de un "0", la cadena es inválida y se rechaza.
- Si se alcanza el blanco (□) en q_1 o q_2, la cadena es aceptada.

2.5.4 Funcionamiento durante la simulación

La simulación se realizó mediante las opciones:

Input → Step
o
Input → Fast Run

Durante la ejecución se observa que:

- La cabeza lectora se desplaza de izquierda a derecha.
- La máquina primero valida que exista al menos un "1".
- Luego permite una transición única hacia la fase de ceros.
- No se permite volver a leer "1" después de haber iniciado los ceros.
- La decisión final se toma cuando se encuentra el símbolo blanco (□).

2.5.5 Ejemplos de validación en JFLAP

A continuación, se presentan ejemplos que demuestran la correcta ejecución de la máquina, uno aceptado y otro rechazado.

2.5.5.1 Ejemplo No. 1

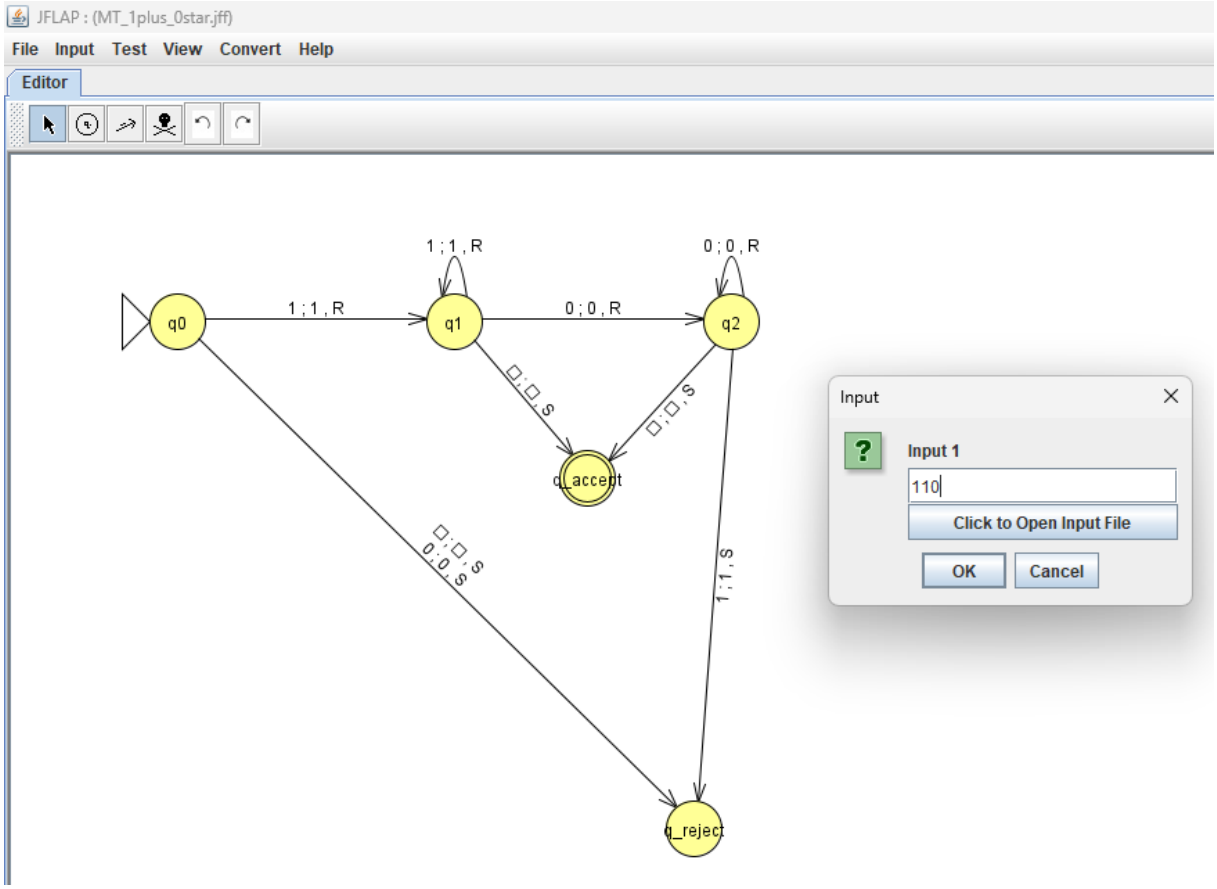
Para la cadena 110:

- q_0 lee 1 → pasa a q_1
- q_1 lee 1 → permanece en q_1
- q_1 lee 0 → pasa a q_2
- q_2 lee □ → pasa a q_accept → ACEPTA

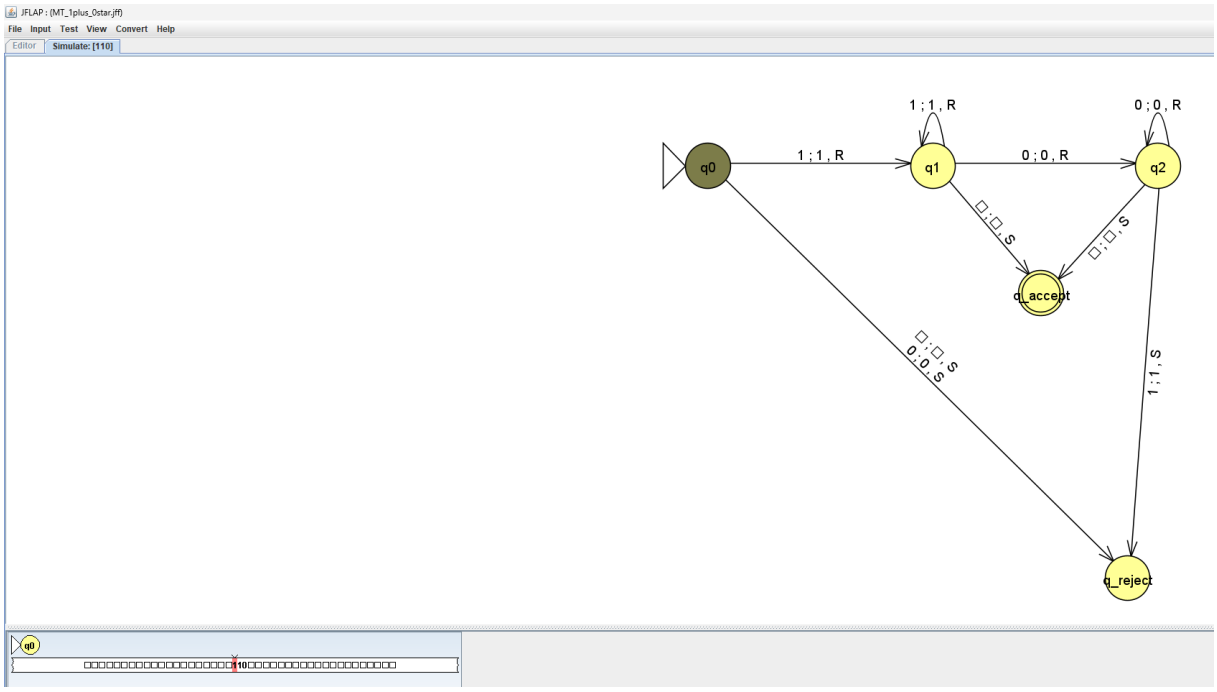
La cadena cumple el patrón: uno o más unos seguidos de cero o más ceros.

A continuación, las imágenes respectivas del proceso:

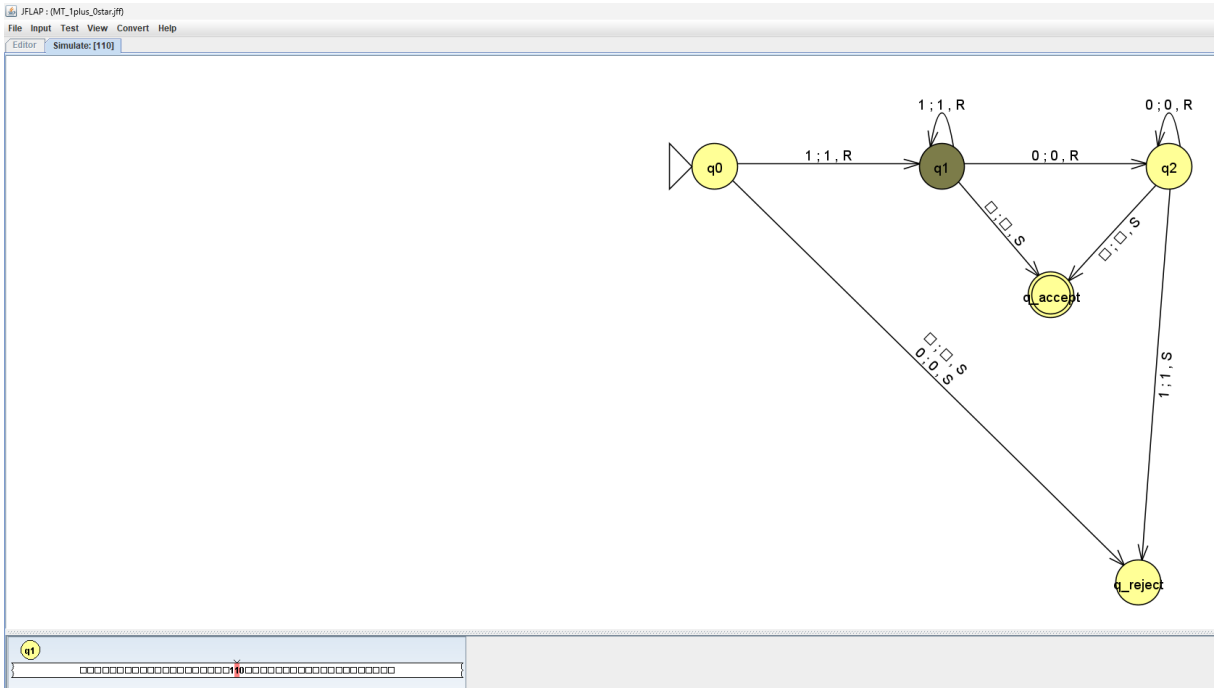
2.5.5.1.1 Imagen 1:



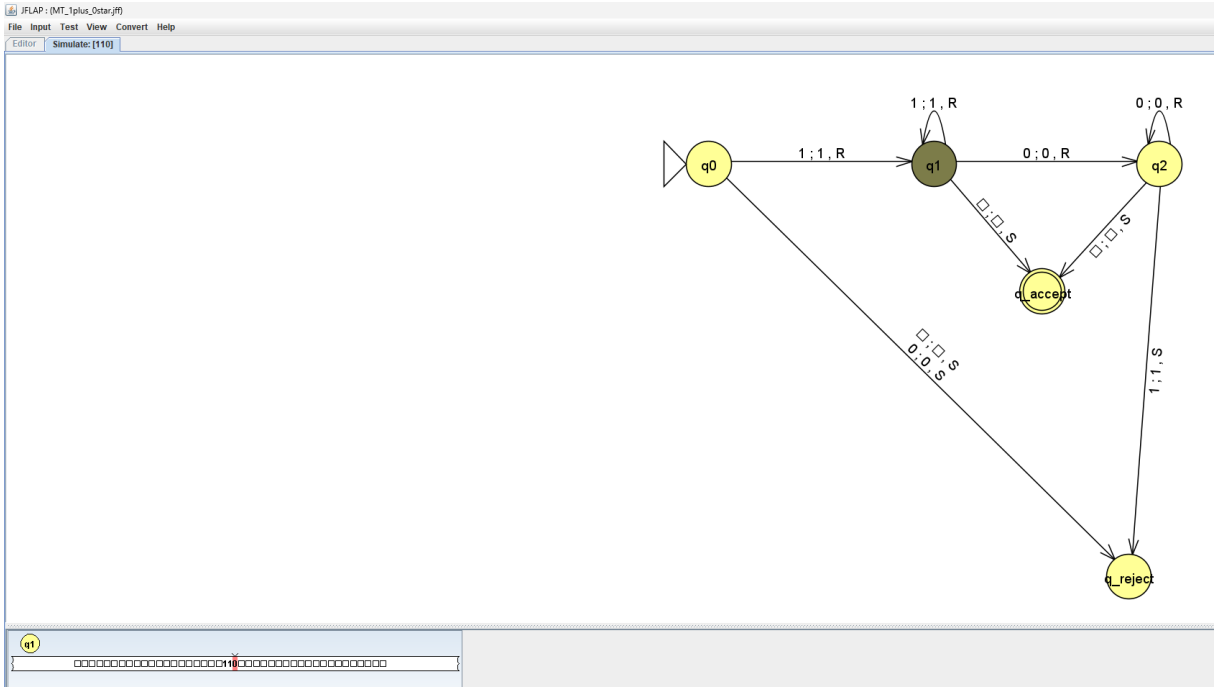
2.5.5.1.2 Imagen 2:



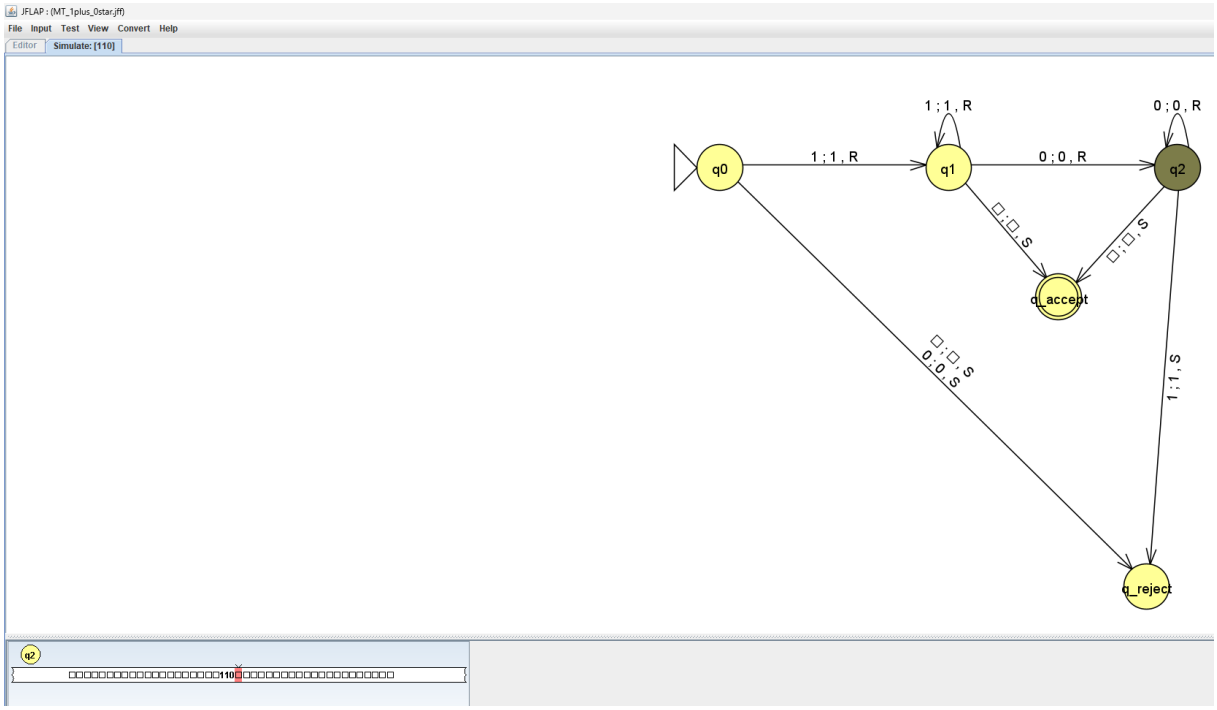
2.5.5.1.3 Imagen 3:



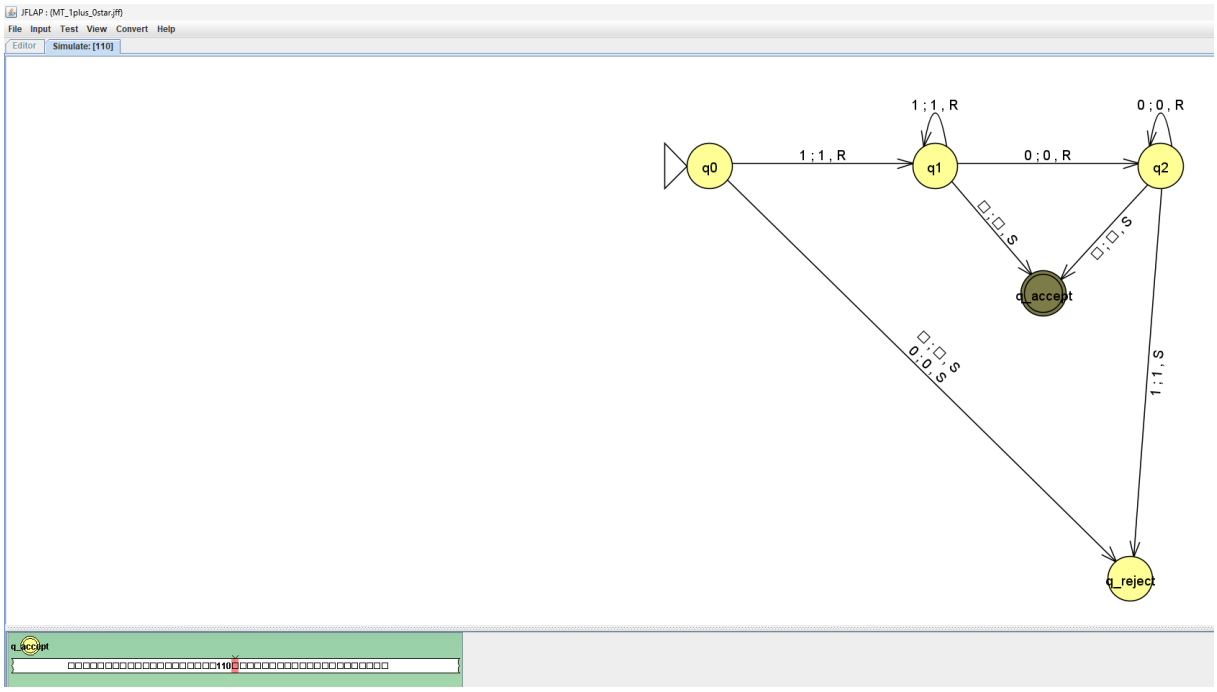
2.5.5.1.4 Imagen 4:



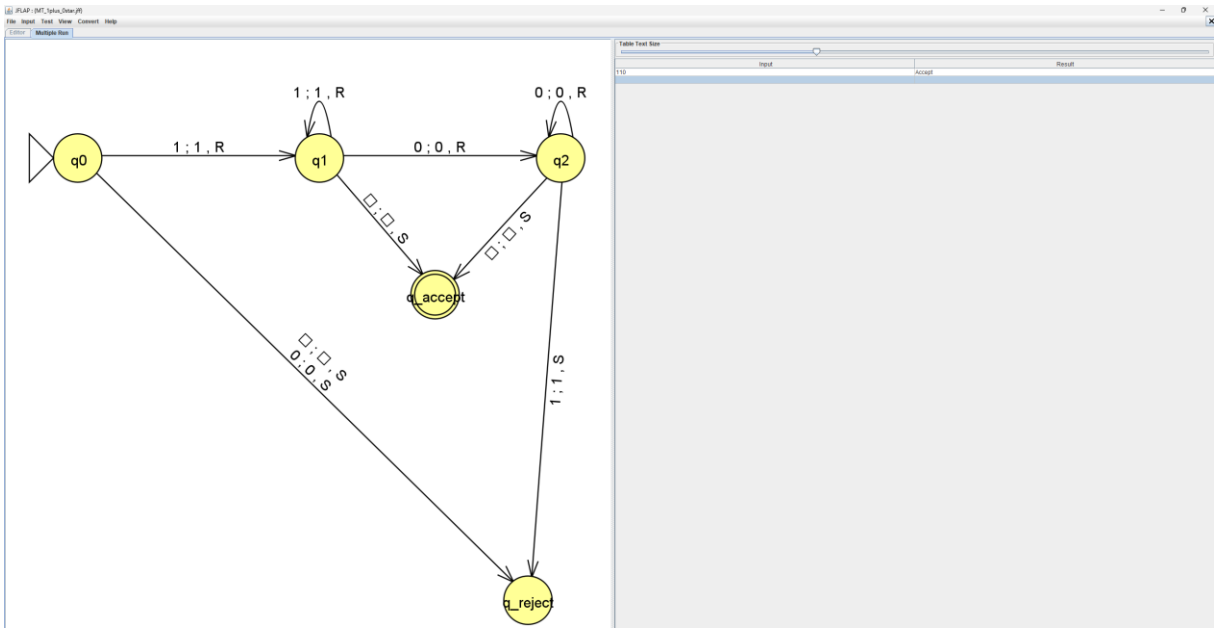
2.5.5.1.5 Imagen 5:



2.5.5.1.6 Imagen 6:



2.5.5.1.7 Imagen 7:



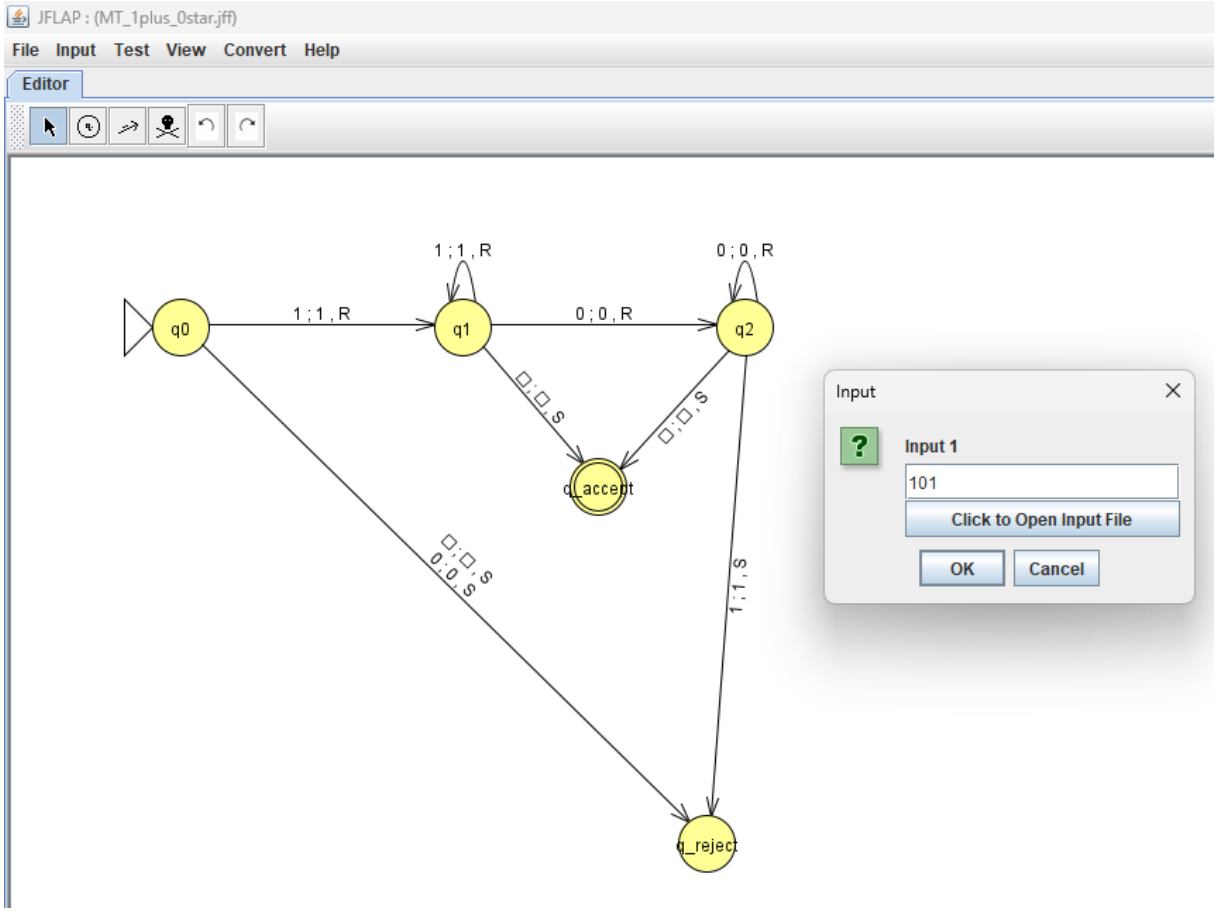
2.5.5.2 Ejemplo No. 2

Para la cadena 101:

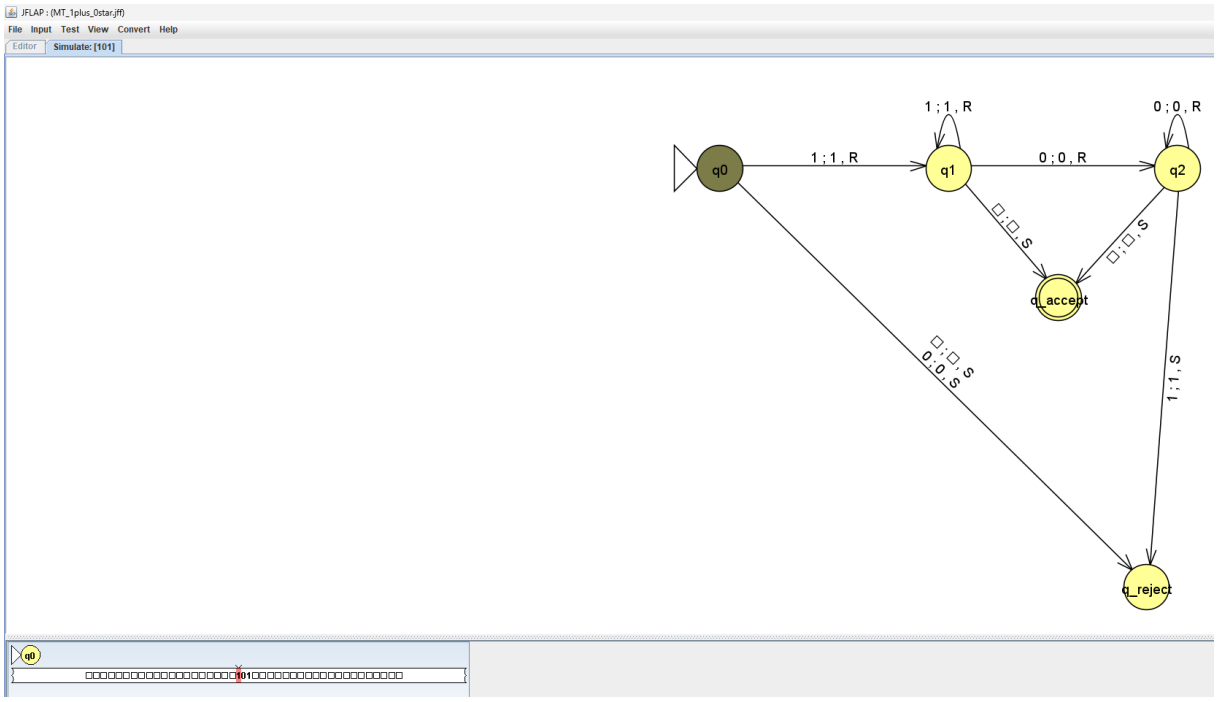
- q_0 lee 1 → pasa a q_1
- q_1 lee 0 → pasa a q_2
- q_2 lee 1 → pasa a q_reject → RECHAZA

El símbolo “1” aparece después de haber comenzado el bloque de ceros, lo cual viola la estructura del lenguaje $1{+}0*$. A continuación, las imágenes respectivas del proceso:

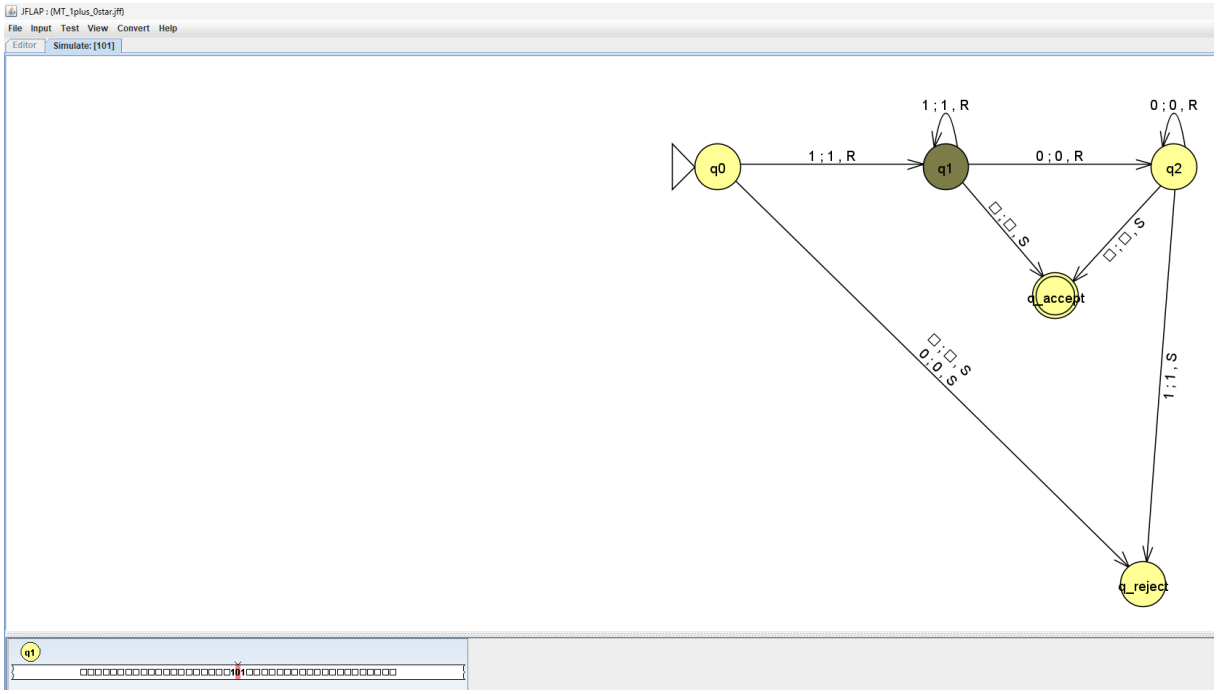
2.5.5.2.1 Imagen 1:



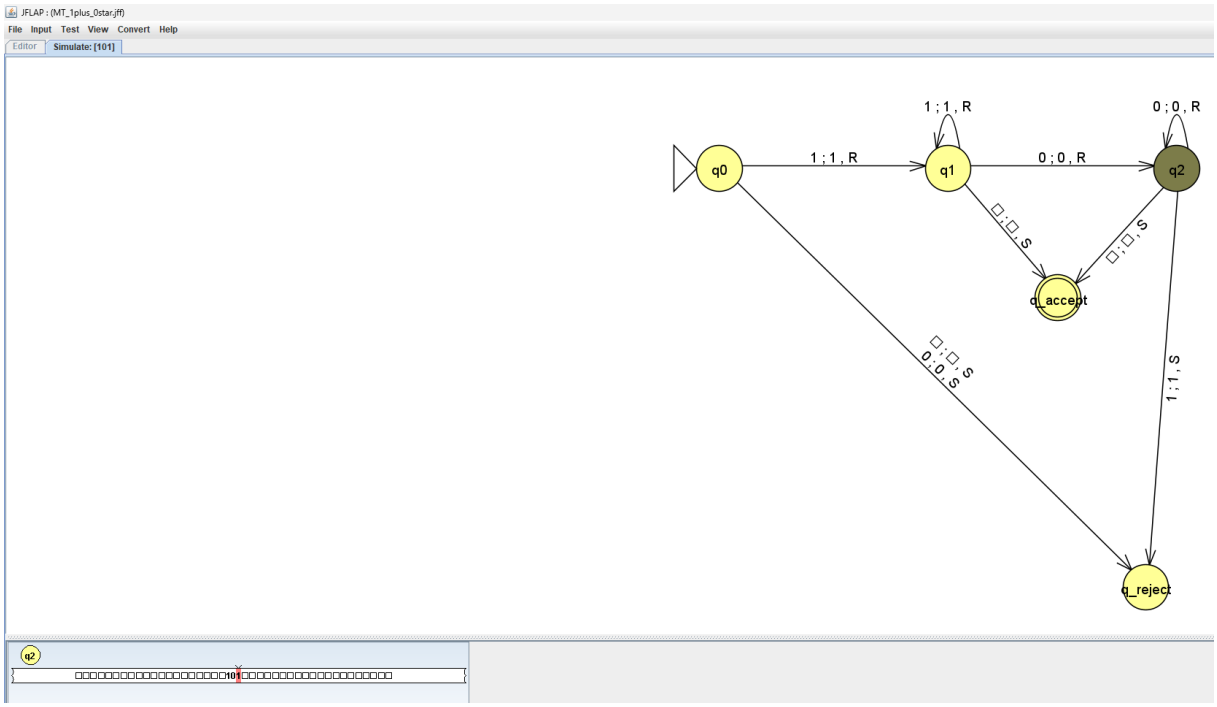
2.5.5.2.2 Imagen 2:



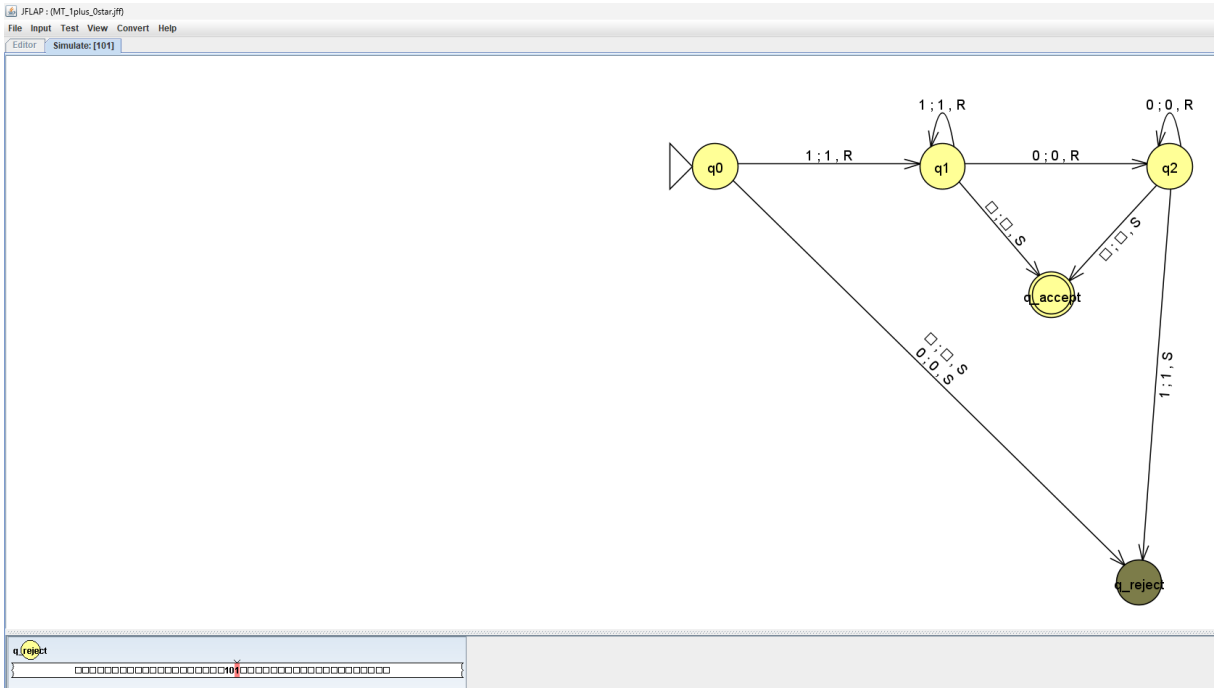
2.5.5.2.3 Imagen 3:



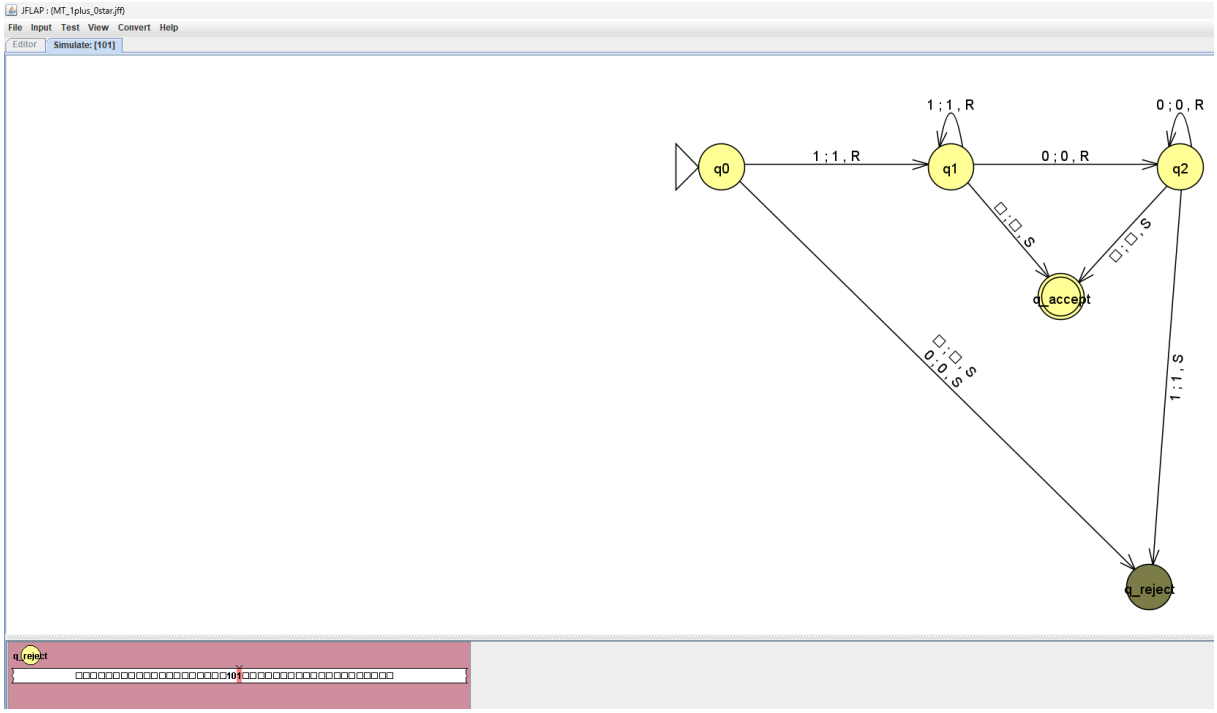
2.5.5.2.4 Imagen 4:



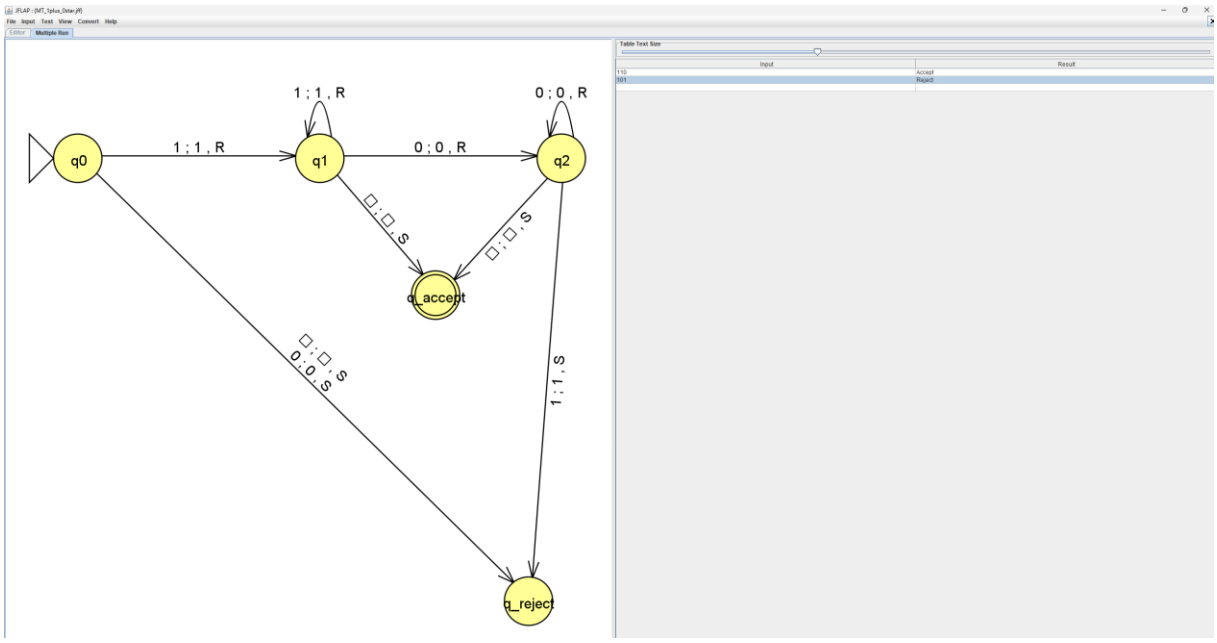
2.5.5.2.5 Imagen 5:



2.5.5.2.6 Imagen 6:



2.5.5.2.7 Imagen 7:



Las simulaciones realizadas confirmaron que la implementación en JFLAP coincide exactamente con el comportamiento esperado según la definición formal, la tabla de transiciones y el diagrama de estados, validando así el correcto funcionamiento de la Máquina de Turing diseñada para reconocer el lenguaje $1 + 0^*$.

CONCLUSIONES DE LA ACTIVIDAD

Para concluir el presente trabajo, se puede afirmar que el desarrollo del laboratorio permitió comprender de manera práctica el funcionamiento y alcance de las Máquinas de Turing como modelo formal de computación. A través del diseño, formalización e implementación en JFLAP, se evidenció cómo este modelo abstracto puede utilizarse para reconocer lenguajes formales específicos mediante un conjunto bien definido de estados y transiciones.

A diferencia de un enfoque exclusivamente teórico, en el cual las máquinas se describen únicamente mediante su séptupla formal y su función de transición, la implementación práctica permitió visualizar el comportamiento dinámico del modelo. Cada ejercicio fue desarrollado de manera estructurada: primero se analizó el problema, luego se diseñó la estrategia de solución, posteriormente se definieron los estados y transiciones formales, y finalmente se validó el funcionamiento mediante simulaciones en JFLAP. Este proceso progresivo facilitó la comprensión del rol que desempeña cada estado dentro de la máquina y permitió verificar la coherencia entre la definición formal y la ejecución real.

En el primer ejercicio, correspondiente al lenguaje:

$$\mathbf{L = \{ w \in \{0, 1\}^* \mid \text{ el numero de símbolos 1 en w es par } \}}$$

Se demostró cómo una Máquina de Turing puede utilizar sus estados como mecanismo de memoria finita para llevar el control de una propiedad específica de la cadena, en este caso, la paridad. Mediante la alternancia entre los estados que representan “par” e “impar”, la máquina logra determinar correctamente si la cantidad total de unos es par al finalizar la lectura de la entrada. Este ejercicio permitió comprender cómo los estados pueden almacenar información lógica sin necesidad de modificar la cinta ni utilizar símbolos auxiliares adicionales.

En el segundo ejercicio, correspondiente al lenguaje:

$$\mathbf{L = \{ 1^n 0^m \mid n \geq 1, m \geq 0 \}}$$

Se diseñó una Máquina de Turing capaz de verificar que la cadena cumpla estrictamente el patrón “uno o más unos seguidos de cero o más ceros”. A través de una estructura organizada en fases (lectura de unos y posteriormente lectura de ceros), la máquina valida que no exista ningún símbolo fuera del orden

establecido. Este ejercicio permitió reforzar la importancia del control estructural de la entrada y evidenció cómo una Máquina de Turing puede reconocer lenguajes definidos por restricciones de forma y orden.

Un aspecto fundamental evidenciado en este laboratorio es que, aunque ambos lenguajes analizados pertenecen a la clase de los lenguajes regulares (y podrían ser reconocidos por autómatas finitos), la implementación mediante Máquinas de Turing permitió profundizar en la comprensión del modelo computacional más general. Esto demuestra que la Máquina de Turing no solo es capaz de reconocer lenguajes complejos, sino que también puede implementar de manera formal procedimientos que podrían resolverse con modelos más simples.

Asimismo, el uso de JFLAP como herramienta de simulación resultó esencial para observar el desplazamiento de la cabeza lectora, la evolución de los estados y el momento exacto en que se produce la aceptación o el rechazo. La experimentación con múltiples cadenas de prueba permitió validar rigurosamente cada diseño y asegurar que las transiciones definidas correspondieran exactamente con el comportamiento esperado según la especificación formal.

En síntesis, esta actividad permitió consolidar la comprensión de las Máquinas de Turing como modelo formal universal, así como fortalecer la capacidad de diseñar sistemas computacionales estructurados a partir de una especificación matemática precisa. Más allá de los resultados obtenidos, el laboratorio evidenció la importancia de la coherencia entre la definición formal, el diagrama de estados y la implementación práctica, demostrando que la rigurosidad en el diseño es fundamental para garantizar el correcto funcionamiento de un sistema de cómputo formal.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada en este desarrollo:

- Tema 1. Conceptos matemáticos utilizados. Informática Teórica (COLGII) - PER 15746 - Enero 2026.
- Tema 2. Lenguajes y gramáticas formales. Informática Teórica (COLGII) - PER 15746 - Enero 2026.
- Tema 3. Introducción a las máquinas de Turing. Informática Teórica (COLGII) - PER 15746 - Enero 2026.
- Tema 4. Extensiones para las máquinas de Turing. Informática Teórica (COLGII) - PER 15746 - Enero 2026.
- Tema 5. Máquinas de Turing restringidas. Computadoras. Informática Teórica (COLGII) - PER 15746 - Enero 2026.
- Clases virtuales con el profesor Ing. Rogerio Orlando Beltrán Castro.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). Introduction to automata theory, languages, and computation (3rd ed.). Pearson.
- Sipser, M. (2013). Introduction to the theory of computation (3rd ed.). Cengage Learning.
- Lewis, H. R., & Papadimitriou, C. H. (1997). Elements of the theory of computation (2nd ed.). Prentice Hall.
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, 42(2), 230–265. https://doi.org/10.1112/plms/s2-42.1.230
- Rodger, S. H., & Finley, T. W. (2006). JFLAP: An interactive formal languages and automata package. Jones & Bartlett Learning.
- Rodger, S. H. (s.f.). JFLAP – Java Formal Languages and Automata Package. Duke University. http://www.jflap.org

AGRADECIMIENTO

Finalmente, deseo expresar mi más sincero agradecimiento al profesor Ing. Rogerio Orlando Beltrán Castro, por los conocimientos, orientación y acompañamiento brindados durante el desarrollo de esta actividad. Sus explicaciones en el área de teoría de la computación y lenguajes formales fueron fundamentales para comprender de manera clara el funcionamiento de las Máquinas de Turing y su aplicación en el reconocimiento de lenguajes formales.

Gracias a los conceptos trabajados en clase, fue posible abordar este laboratorio de manera estructurada, iniciando con el análisis formal de los lenguajes propuestos, continuando con la definición rigurosa de estados y funciones de transición, y culminando con la implementación práctica en la herramienta JFLAP. Este proceso permitió entender cómo una Máquina de Turing puede utilizar sus estados como mecanismo de control para verificar propiedades específicas, como la paridad de símbolos o el cumplimiento de un patrón estructural determinado.

Asimismo, la práctica realizada fortaleció la capacidad de traducir una definición matemática formal en un diseño funcional dentro de un simulador, garantizando coherencia entre la teoría y la implementación. Sin duda, esta experiencia contribuyó significativamente a consolidar mi comprensión de los fundamentos de la computabilidad y del diseño formal de sistemas de cómputo.

¡¡¡Mil gracias profesor!!!

Respetuosamente,

Alejandro De Mendoza